Bachelor's Thesis Computing Science




Radboud University Nijmegen

---

**Formalising Modal Embeddings of Call-by-Name and Call-by-Value**

---

*Author:*
Floris Reuvers
s1096976

*First supervisor/assessor:*
dr. N.M. van der Weide

*Second assessor:*
dr. E.G.M. Hubbers

June 26, 2025

**Abstract**

This thesis presents a formalisation of the boxed $\lambda$-calculus ($\lambda_b$) in the formal proof assistant Agda. This calculus is a refinement of $\lambda_\Box$, a modal $\lambda$-calculus that is a fragment of IS4. Girard's and Gödel's embeddings are translations from the call-by-name $\lambda$-calculus ($\lambda_n$) and the call-by-value $\lambda$-calculus ($\lambda_v$) into the shared target, the call-by-box $\lambda$-calculus ($\lambda_b$). We formalise $\lambda_b$ and these embedding in Agda. We then prove that Girard's embedding preserves reduction and evaluation, and that Gödel's embedding preserves reduction as well.

# Contents


**1 Introduction** **3**

**2 Lambda Calculus** **5**
2.1 Introduction to the $\boldsymbol{\lambda}$-calculus . . . 5
2.2 Well-Typedness . . . 6
2.3 $\boldsymbol{\beta}$-Reduction . . . 7
2.3.1 Free and Bound Variables . . . 7
2.3.2 Substitution . . . 8
2.3.3 $\boldsymbol{\alpha}$-equivalence . . . 10
2.3.4 $\boldsymbol{\beta}$-Reduction . . . 10
2.4 Call-by-name $\boldsymbol{\lambda}$-calculus . . . 11
2.5 Call-by-value $\boldsymbol{\lambda}$-calculus . . . 12

**3 Call-by-box Lambda Calculus** **15**
3.1 Specification of $\boldsymbol{\lambda_b}$ . . . 15
3.1.1 Intuitionistic logic IS4 . . . 15
3.1.2 Terms and Types of $\boldsymbol{\lambda_b}$ . . . 16
3.2 Substitution in $\lambda_b$ . . . 18

**4 Embeddings in $\boldsymbol{\lambda_b}$** **22**
4.1 Girard's embedding . . . 22
4.2 Gödel's Embedding . . . 24

**5 Formalisation in Agda** **28**
5.1 Intro to Agda . . . 28
5.1.1 Data Types and Functions . . . 28
5.1.2 Hole-Based Programming . . . 30
5.2 Challenges of the Formalisation . . . 32
5.2.1 De Bruijn Indices . . . 32
5.2.2 Well-typed Terms . . . 33
5.2.3 Substitution . . . 37
5.2.4 Weakening and Renaming . . . 40
5.3 Overview of the Formalisation . . . 41
5.3.1 File Structure . . . 42

5.3.2 Helper Lemmas . . . . . 42
5.3.3 Proofs of the Main Propositions . . . . . 44

**6 Related Work** **46**

**7 Conclusions and Future Work** **48**
7.1 Conclusions . . . . . 48
7.2 Future Work . . . . . 49

# Chapter 1

# Introduction

One of the simplest, yet most powerful, models of computation is the $\lambda$-calculus. It serves as the foundation of many programming languages and proof assistants, such as Haskell and Agda. The two main well-known evaluation strategies for the $\lambda$-calculus are *call-by-name* (cbn) and *call-by-value* (cbv). Call-by-name does not evaluate an argument to a value until it is needed, while call-by-value evaluates an argument to a value before passing it to a function. These are two very different approaches and depending on the context, one may be more suitable than the other.

Cbn is more efficient when some arguments are not used inside a function, as it avoids unnecessary computation. However, if an argument is used multiple times, cbv is preferable, as it avoids recomputing the argument each time it is used. Lazy evaluation, which is used in Haskell, is an optimisation of cbn, where computations are shared. In contrast to cbn, lazy evaluation avoids recomputing an argument multiple times, as it stores and reuses the result of the computation. Although lazy evaluation might seem like a good compromise, cbv is sometimes preferable because it is more predictable. Therefore, a $\lambda$-calculus that can express both lazy evaluation and cbv is ideal. This thesis focuses on the unification of cbn and cbv, and not on lazy evaluation, as it is more complex. A unifying framework allows for the creation of a programming language or proof assistant that is more flexible in evaluation strategies. While cbv can be simulated in Haskell, it requires some workarounds and support for cbv is not built into the language.

Many approaches to unifying cbn and cbv have been proposed. Ehrhard and Guerrieri [14] proposed a linear logic approach, just like the one by Maraist et al., [24], while Hatcliff and Danvy [19] proposed a thunk-based approach. However, we focus on the modal approach, introduced by Espírito Santo et al. [15]. This approach is based on the modal $\lambda$-calculus $\lambda_{\Box}$, which is a fragment of Bierman and de Paiva's intuitionistic modal logic (IS4) [8]. The proposed call-by-box $\lambda$-calculus ($\lambda_b$) is a refinement of $\lambda_{\Box}$, which has simpler typing rules and a more straightforward reduction strategy.

This thesis aims to formalise this approach in Agda, which is a dependently typed programming language and proof assistant. Although $\lambda_b$ is discussed in the literature, it has not yet been formalised in a proof assistant before. Given the extra complexity of the $\lambda_b$ in comparison to the standard $\lambda$-calculus, it is important to formalise it to ensure that the properties of $\lambda_b$ are correct and machine-checked. Therefore, the main contribution of this thesis is as follows:

> A formalisation of the unification of call-by-name and call-by-value using modal logic in Agda?

In $\lambda_b$, cbn and cbv can be simulated by using Girard's and Gödel's embeddings, respectively. These are embeddings from the standard $\lambda$-calculus to $\lambda_b$, preserving both reduction and evaluation. Besides $\lambda_b$, this thesis also formalises both embeddings, and we prove the following in Agda:

- Girard's embedding preserves reduction. Proposition 4.5
- Girard's embedding preserves evaluation. Proposition 4.6
- Gödel's embedding preserves reduction. Propositions 4.11 and 4.13

We did not formally prove that Gödel's embedding preserves evaluation, but a less formal proof is given by [15]

We now give a brief overview of the chapters in this thesis. Chapter 2 introduces the $\lambda$-calculus, with substitution, $\beta$-reduction and evaluation relations. The next chapter, Chapter 3, introduces $\lambda_b$, with substitution, $\beta$-reduction and its evaluation relations. We then discuss the two embeddings in Chapter 4, where we also provide the propositions about the embeddings. Chapter 5 explains the basic Agda concepts, the challenges we faced while formalising $\lambda_b$, the embeddings, and how we solved them. Finally, Chapter 6 gives an overview of related work in the field, and Chapter 7 concludes the thesis.

# Chapter 2

# Lambda Calculus

This chapter provides a detailed and formal description of the $\lambda$-calculus, defined by Church in [10]. We define a formal grammar of the $\lambda$-calculus in the same way as [15] and give examples of some terms in the $\lambda$-calculus. We mostly follow the notation of $\lambda$-terms in [21]. Only for substitution, we use a different notation, which we discuss later in Section 2.3.2. Then we explain $\beta$-reduction, the only way to reduce $\lambda$-terms [10], after which we define call-by-name evaluation and call-by-value evaluation [28].

## 2.1 Introduction to the λ-calculus

The grammar of the lambda calculus is as follows:

**Definition 2.1. λ-calculus.** Terms of the $\lambda$-calculus are given by:
$M, N, P, Q \quad ::= \quad x \mid \lambda x.M \mid MN$

Terms are denoted by $M, N, P$ or $Q$ and can either be of the form $x$, $\lambda x.M$ or $MN$:

- $x$ is a variable, which is a symbol that represents an input. Variables are undetermined, and they represent other terms.
- $\lambda x.M$ is an abstraction. An abstraction is an anonymous function, where $x$ is the parameter and $M$ is the body of the function.
- $MN$ is a function application, where $M$ and $N$ are terms.

The $\lambda$-term $\lambda x.x$ is an abstraction. This specific abstraction is called the identity function and has one input parameter, namely $x$, and returns the input $x$. The body of the function is also $x$, and the function is usually abbreviated as $\mathbf{I}$ as done in [12]. The $\lambda$-term $(\lambda x.x)y$ is a function application, which states that the identity function is applied to the variable $y$.

## 2.2 Well-Typedness

This section discusses the concept of well-typedness in the context of lambda calculus. We first discuss types, typing judgements and typing rules. We then explore how to use these rules to show that a $\lambda$-term is well-typed.

Types of $\lambda$-calculus are defined in Definition 2.2, and are either $X$, which is the base type, or a function type, $A \to A'$ with $A$ and $A'$ being types in $\lambda$-calculus. This formulation follows from Church's "simple typed $\lambda$ calculus", introduced in [11]. In this thesis, we use the Curry-style notation where we write terms without explicit types. This notation is formulated by Curry, and the notation used here is discussed in [6].

**Definition 2.2. Types of $\lambda$-calculus** are defined via:

$$A \quad ::= \quad X \mid \ A \to A'$$

We define a context, usually denoted by $\Gamma$, by a list of variable-type pairs, where each variable is associated with a type. A context is written as $\Gamma = x_1 : A_1, \ldots, x_n : A_n$, where each $x_i$ is a variable and each $A_i$ is a type. A typing judgement is of the form $\Gamma \vdash M : A$, and it means that we can derive that $M$ has type $A$ from the assumptions in context $\Gamma$.

The typing rules are defined in Definition 2.3, and are from [6]. The Id rule states that if $x$ has type $A$ in the context, we can conclude that $x$ has type $A$. The $\to$-I rule states that if we have a term $M$ of type $A'$ in the context $\Gamma, x : A$, we can conclude that from context $\Gamma$ it follows that $\lambda x.M$ has type $A \to A'$. The $\to$-E rule states that if we have a term $M$ of type $A \to A'$ and a term $N$ of type $A$, then the application $MN$ has type $A'$.

**Definition 2.3. The Typing rules** of the $\lambda$-calculus are given by:

$$\frac{}{\Gamma, x : A \vdash x : A}\ \text{Id}$$

$$\frac{\Gamma, x : A \vdash M : A'}{\Gamma \vdash \lambda x.M : A \to A'}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : A \to A' \qquad \Gamma \vdash N : A}{\Gamma \vdash MN : A'}\ \to\text{-E}$$

A $\lambda$-term is considered to be *well-typed* if we can derive a typing judgement with the typing rules. More concretely, this means that well-typed $\lambda$-terms can be assigned a type. In Example 2.4, we show that $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$ is well-typed by giving the full typing derivation.

**Example 2.4. Well-typedness of $\mathbf{I}(\lambda x.\mathbf{I}x)$.** The typing derivation of $\lambda$-term $\mathbf{I}(\lambda x.\mathbf{I}x)$ is given by the tree below.

$$\dfrac{\dfrac{\dfrac{}{y : A \to A \vdash y : (A \to A)}\,\text{Id}}{\vdash \lambda y.y : (A \to A) \to (A \to A)}\,{\to}\text{-I} \qquad \dfrac{\dfrac{\dfrac{\dfrac{}{x : A, z : A \vdash z : A}\,\text{Id}}{x : A \vdash \lambda z.z : A \to A}\,{\to}\text{-I} \qquad \dfrac{}{x : A \vdash x : A}\,\text{Id}}{x : A \vdash \mathbf{I}x : A}\,{\to}\text{-E}}{\vdash \lambda x.\mathbf{I}x : A \to A}\,{\to}\text{-I}}{\vdash \mathbf{I}(\lambda x.\mathbf{I}x) : A \to A}\,{\to}\text{-E}$$

In contrast to the $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))$, the lambda term $MM$ cannot be assigned a type. Since $MM$ is an application, we can only apply the $\to$-E rule, which would give a contradiction. Given the $\to$-E rule, $M$ needs to be some kind of function with type $A \to A'$. However, given the $\to$-E rule, $M$ also needs to be the input of the function with type $A$. This results in a contradiction, as $M$ cannot be both of type $A \to A'$ and type $A$.

## 2.3 $\beta$-Reduction

To include a notion of computation to the $\lambda$-calculus, we discuss $\beta$-reduction. This notion of computation was introduced by Church in [10] and the definition of $\beta$-reduction we use is by [6]. This reduction relation is the only way to simplify terms in the $\lambda$-calculus. With $\beta$-reduction, we apply a $\lambda$ abstraction to a $\lambda$-term (argument) by replacing the bound variable of the abstraction with that argument. To do this, we give definitions of substitution, free and bound variables similar to the definitions in [6].

### 2.3.1 Free and Bound Variables

Let us first define free and bound variables [28, 6]. Let $P$ be any term in the $\lambda$-calculus, $FV(P)$ be the set that contains all free variables in $P$ and $FB$ be the set that contains all bound variables in $P$. We define both $FV$ and $BV$ inductively on $P$ by Definition 2.5 and Definition 2.6. Since $P$ can only be a variable, an abstraction or a function application, $P$ can only be of the form $x$, $\lambda x.M$ and $MN$. Therefore, we define $FV$ and $BV$ for $x$, $\lambda x.M$ and $MN$.

**Definition 2.5. Free Variables** are given by the function $FV$ as follows:

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.M) &= FV(M) \backslash \{x\} \\ FV(MN) &= FV(M) \cup FV(N) \end{aligned}$$

Let us now explain $FV$. The variable $x$ in the $\lambda$-term $x$ is free. All free variables in an abstraction are those that are not the parameters of the abstraction. All free variables in an application $MN$ are all free variables of $M$ plus the free variables of $N$.

**Definition 2.6. Bound Variables** are given by the function $BV$ as follows:

$$\begin{aligned} BV(x) &= \emptyset \\ BV(\lambda x.M) &= BV(M) \cup \{x\} \\ BV(MN) &= BV(M) \cup BV(N) \end{aligned}$$

Let us now explain $BV$. The variable $x$ in the $\lambda$-term $x$ is a free variable, and the set of bound variables is empty. An abstraction $\lambda x.M$ binds the variable $x$. So the set of bound variables in an abstraction is the set of bound variables in $M$ plus the variable $x$. For an application $MN$, the bound variables are all bound variables of both $\lambda$-term $M$ and $\lambda$-term $N$.

We give two examples to illustrate the definitions of free and bound variables. We show that $\lambda$-term $\lambda x.yx$ has one free variable, $y$ and one bound variable $x$.

**Example 2.7. Free and Bound variables** of $\lambda$-term $\lambda x.yx$ are found by the following calculations:

$$\begin{aligned} FV(\lambda x.yx) &= FV(yx)\backslash\{x\} \\ &= (FV(x) \cup FV(y))\backslash\{x\} \\ &= (\{x\} \cup \{y\})\backslash\{x\} \\ &= \{x, y\}\backslash\{x\} \\ &= \{y\} \end{aligned}$$

$$\begin{aligned} BV(\lambda x.xy) &= BV(xy) \cup \{x\} \\ &= (BV(x) \cup BV(y)) \cup \{x\} \\ &= (\emptyset \cup \emptyset) \cup \{x\} \\ &= \{x\} \end{aligned}$$

The following example demonstrates that using two different variables with the same name in a $\lambda$-term is quite tricky: $(\lambda x.x)x$. By the definitions of $FV$ and $BV$, we can reason that $FV((\lambda x.x)x) = \{x\}$ and $BV((\lambda x.x)x) = \{x\}$. While these sets are the same, they do not refer to the same $x$. This is quite confusing, and we prefer to write $(\lambda y.y)x$ instead, where $y$ is a new "fresh" variable with a different name. We now have $FV((\lambda y.y)x) = \{x\}$ and $BV((\lambda y.y)x) = \{y\}$. These $\lambda$-terms are equivalent by the notion of $\alpha$-equivalence, which states that $\lambda$-terms that differ only by the names of bound variables are considered the same. We discuss this in more detail after we define substitution.

### 2.3.2 Substitution

Keeping the definitions of free and bound variables in mind, we now define substitution. In this thesis, substitution is denoted as $P[x \mapsto Q]$ and is

defined inductively on $P$ (see Definition 2.8). Essentially, it replaces all free occurrences of the variable $x$ in the $\lambda$-term $P$ by $\lambda$-term $Q$. The original notation of the substitution by Curry, Feys, and Craig in [13], denoting the substitution of $Q$ for $x$ in $P$, is given by $[Q/x]P$. However, we chose not to use this original notation, as we think that $P[x \mapsto Q]$ for substituting $Q$ for $x$ in $P$ is clearer and more intuitive. This alternative notation was used by Hanne Riis Nielson and Flemming Nielson in [26]. We adopt the definition of substitution from [28], defined by Plotkin.

**Definition 2.8. Substitution** in the $\lambda$-calculus:

$$\begin{aligned} x[x \mapsto Q] &= Q \\ y[x \mapsto Q] &= y \text{ if } (x \neq y) \\ (MN)[x \mapsto Q] &= M[x \mapsto Q]N[x \mapsto Q] \\ (\lambda x.M)[x \mapsto Q] &= \lambda x.M \\ (\lambda y.M)[x \mapsto Q] &= \lambda z.M[y \mapsto z][x \mapsto Q] \text{ if } (x \neq y) \end{aligned}$$

with $z$ a variable defined by:

1. If $x \notin FV(Q)$ or $y \notin FV(M)$ then $z = y$
2. Otherwise, $z$ can be any variable such that $z \notin FV(Q)$ or $z \notin FV(M)$

The collection of variables is infinite, so the variable $z$ always exists. Choosing a fresh variable for $z$ always satisfies condition two. It might not be clear why we need to use the variable $z$ and one might expect the following rule to be sufficient.

$$(\lambda y.M)[x \mapsto Q] = \lambda z.M[x \mapsto Q] \text{ if } (x \neq y)$$

However, the following example illustrates why the more complicated rule is necessary. Consider the substitution $(\lambda y.xy)[x \mapsto y]$. The free variable of $\lambda$-term $\lambda y.xy$ is $x$ and the bound variable is $y$. If we use the simple rule above, we get that $(\lambda y.xy)[x \mapsto y] = \lambda y.yy$, the meaning of the $\lambda$-term changes. However, the free variable we substituted, is not free anymore. The variable $y$ is not a free variable in the term that we substituted $y$ in. Therefore, we change the bound variable $y$ in $\lambda y.xy$ to a new variable $z$. We can pick $z$ as $z$ is not a free variable in $xy$. Following the more complex rule, we get:

$$\begin{aligned} (\lambda y.xy)[x \mapsto y] &= \lambda z.(xy)[y \mapsto z][x \mapsto y] \\ &= \lambda z.yz \end{aligned}$$

### 2.3.3 $\alpha$-equivalence

Here, we define $\alpha$-equivalence. We use the definition by Plotkin in [28]. The idea is that renaming bound variables does not change a $\lambda$-term. For example, the $\lambda$-terms $\lambda x.x$ and $\lambda y.y$ are equivalent ($\lambda x.x \equiv_\alpha \lambda y.y$) since renaming all bound variables can result in identical $\lambda$-terms. The $\lambda$-terms $\lambda x.x$ and $\lambda y.y$ represent the same idea. We define the relation $P \equiv_\alpha P'$ inductively on $P$ and $P'$ in Definition 2.9.

**Definition 2.9. $\alpha$-equivalence** of $\lambda$-terms:

$$
\begin{array}{ll}
x \equiv_\alpha y & \text{if } x = y \\
MN \equiv_\alpha M'N' & \text{if } M \equiv_\alpha M' \text{ and } N \equiv_\alpha N' \\
\lambda x.M \equiv_\alpha \lambda y.M' & \text{if } M \equiv_\alpha M'[y \mapsto x] \text{ with } x = y \text{ or } x \notin FV(M')
\end{array}
$$

The last rule states that abstractions $\lambda x.M$ and $\lambda y.M'$ are equivalent if $M$ and $M'$ are equivalent after substituting $y$ for $x$ in $M'$. However, $x$ should be equal to $y$ or $x$ should not be a free variable in $M'$.

**Example 2.10. $\alpha$-equivalence** of $\lambda$-terms. Here, we demonstrate $\alpha$-equivalence of abstractions:

- $\lambda x.zx$ is equivalent to $\lambda y.zy$, since $zy[y \mapsto x] = zx \equiv_\alpha zx$ and $x$ is not a free variable in $zy$.
- $\lambda z.zx$ is not equivalent to $\lambda y.zy$, since $z$ is a free variable in $zy$ and $x$ is not equal to $y$.

### 2.3.4 $\beta$-Reduction

Now, we are ready to discuss $\beta$-reduction. In the $\lambda$-calculus, there is only one way to simplify terms, which is $\beta$-reduction. We use the definition of $\beta$-reduction discussed in [6] and Definition 2.11 defines $\beta$-reduction with $M$ and $N$ being terms in the $\lambda$-calculus.

**Definition 2.11. $\beta$-reduction** in $\lambda$-calculus:

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta)$$

A $\lambda$-term of the form $(\lambda x.M)N$ can be reduced by substituting $x$ by $N$ in $M$. To apply $\beta$-reduction to subterms, we define closure rules. We use closure rules similar to how Espírito Santo, Pinto, and Uustalu use them in [15]. Closure rules are also discussed in [6] and [28]. For example, the following reduction would not be possible: $\lambda x.(\lambda y.y)x \to \lambda x.x$. Therefore, in Definition 2.12, we define the following closure rules: $\mu$, $\nu$ and $\xi$.

**Definition 2.12. Closure rules for $\beta$-reduction:**

$$\frac{M \to M'}{MN \to M'N}\,(\mu) \qquad \frac{N \to N'}{MN \to MN'}\,(\nu) \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}\,(\xi)$$

Now, we define $\to_\beta$ as $\beta$ closed under $\mu$, $\nu$ and $\xi$. So with $\to_\beta$ we can use $\beta$-reduction on all subterms if the subterm is of the form $(\lambda x.M)N$. A subterm of the form $(\lambda x.M)N$ is called a $\beta$-redex. In one reduction, we may need to use multiple rules. The $\beta$-redex that is reduced is underlined. For instance, we need to use the $\mu$ and $\xi$ rule in the last example of the following reductions:

**Example 2.13. Usage of Closure Rules:**

$$\begin{aligned}
(\underline{\mathbf{II}})(\mathbf{II}) &\to_\beta \mathbf{I}(\mathbf{II}) && \beta \text{ with } \mu \text{ rule}\\
(\mathbf{II})(\underline{\mathbf{II}}) &\to_\beta (\mathbf{II})\mathbf{I} && \beta \text{ with } \nu \text{ rule}\\
\lambda x.\underline{\mathbf{I}x} &\to_\beta \lambda x.x && \beta \text{ with } \xi \text{ rule}\\
(\mathbf{I}(\lambda x.\underline{\mathbf{I}x}))(\mathbf{II}) &\to_\beta (\mathbf{I}(\lambda x.x))(\mathbf{II}) && \beta \text{ with } \mu \text{ and } \xi \text{ rule}
\end{aligned}$$

We see that $\beta$-reduction is a non-deterministic relation, which means that there can be multiple ways to reduce a term. However, sometimes we would only like to reduce a term in one way and have a deterministic relation. To achieve this, we can define subrelations that do not allow all closure rules, which we see in Sections 2.4 and 2.5.

## 2.4 Call-by-name $\lambda$-calculus

In this section, we discuss the call-by-name $\lambda$-calculus ($\lambda_n$). The reduction rule that is used in this calculus is the same as the $\beta$ reduction rule. However, we name the rule $\beta_n$ (Definition 2.14) to make it clear that it is the reduction rule of $\lambda_n$. While call-by-name avoids evaluating the argument of a function, arguments are re-evaluated every time they are used in the body of the function. Lazy evaluation (call-by-need) stores the evaluation of each argument, so it does not have to re-evaluate the same argument every time it is used. Therefore, an optimised version of call-by-name evaluation is lazy evaluation, which is used in many programming languages such as Clean or Haskell. Zena et al. [4] define a call-by-need calculus, but for simplicity, we only use $\lambda_n$ in this thesis.

**Definition 2.14. $\beta_n$-reduction** in $\lambda_n$:

$$(\lambda x.M)N \to M[x \mapsto N] \qquad (\beta_n)$$

The relations we define in this thesis are all similar to the relations discussed in [15]. We define the relation $\rightarrow_{\beta_n}$ as $\beta_n$ closed under $\mu$, $\nu$ and $\xi$. The relation $\rightarrow_{\beta_n}$ is the call-by-name reduction relation, which allows for the reduction of $\lambda$-terms under all term formers. Weak reduction, $\rightarrow_w$, is defined as $\beta_n$ closed under $\mu$ and $\nu$. The only restriction of $\rightarrow_w$ is that it cannot reduce $\lambda$-terms under $\lambda$'s. Weak reduction becomes relevant when unifying cbn and cbv. We also define the relation $\rightarrow_n$ as $\beta_n$ closed under $\mu$. This relation $\rightarrow_n$ cannot evaluate the argument of a function, since the $\nu$ rule is not allowed with $\rightarrow_n$. Call-by-name evaluation is thus defined as $\rightarrow_n^*$.

Table 2.1 shows the closure rules for the different reduction relations we have defined thus far. In Example 2.15, we give the complete cbn evaluation of $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. The $\beta$-redex that is reduced is underlined in each step. Note that $\rightarrow_n$ is deterministic, so there is always only one way to apply $\rightarrow_n$ on a $\lambda$-term.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\xi}$ |
|---|---|---|---|---|
| $\rightarrow_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ |
| $\rightarrow_n$ | $\beta_n$ | ✓ | | |
| $\rightarrow_w$ | $\beta_n$ | ✓ | ✓ | |

Table 2.1: Closure Rules for Different Reduction Relations

**Example 2.15. Cbn evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$:**

$$\begin{aligned}(\underline{\mathbf{I}(\lambda x.\mathbf{I}x)})(\mathbf{II}) &\rightarrow_n \underline{(\lambda x.\mathbf{I}x)(\mathbf{II})}\\ &\rightarrow_n \underline{\mathbf{I}(\mathbf{II})}\\ &\rightarrow_n \underline{\mathbf{II}}\\ &\rightarrow_n \mathbf{I}\end{aligned}$$

## 2.5 Call-by-value $\lambda$-calculus

The call-by-value $\lambda$-calculus ($\lambda_v$) is defined by Gordon Plotkin in [28] and this section defines it similarly. We first define the notion of values. Values are $\lambda$-terms that are fully evaluated. That is, values are $\lambda$-terms that cannot be reduced any further and thus do not contain any function applications. We define values in Definition 2.16 as all $\lambda$-terms that are either a variable or an abstraction. We denote values by $V$ or $W$.

**Definition 2.16. Values in $\boldsymbol{\lambda_v}$** are given via:

$$V \quad ::= \quad x \mid \lambda x.M$$

The main idea of $\lambda_v$ is that we only reduce a function application if the argument is a value. The $\beta$-reduction rule is defined in Definition 2.17.

**Definition 2.17. $\beta_v$-reduction** in $\lambda_v$:

$$(\lambda x.M)V \to M[x \mapsto V] \qquad (\beta_v)$$

We define a new closure rule, $\nu_<$ [15], which is similar to the $\nu$ rule but requires the left $\lambda$-term to be a value. In call-by-value evaluation, we first reduce the left $\lambda$-term of a function application, to an abstraction, before reducing the right $\lambda$-term to a value to apply $\beta_v$.

**Definition 2.18. The closure rule $\nu_<$:**

$$\frac{N \to N'}{VN \to VN'}\,(\nu_<)$$

We define call-by-value reduction, $\to_{\beta_v}$, as $\beta_v$ closed under $\mu$, $\nu$ and $\xi$ which allows reduction of $\lambda$-terms under all term formers. The relation $\to_v$ is defined as $\beta_v$, closed under $\mu$ and $\nu_<$. Table 2.2 shows the closure rules for all different reduction relations we have defined thus far.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\nu_<}$ | $\boldsymbol{\xi}$ |
|---|---|---|---|---|---|
| $\to_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ | ✓ |
| $\to_n$ | $\beta_n$ | ✓ | | | |
| $\to_w$ | $\beta_n$ | ✓ | ✓ | | |
| $\to_{\beta_v}$ | $\beta_v$ | ✓ | ✓ | ✓ | ✓ |
| $\to_v$ | $\beta_v$ | ✓ | | ✓ | |

Table 2.2: Closure Rules for Different Reduction Relations

Similar to call-by-name evaluation, call-by-value evaluation is defined as $\to_v^*$. Here the star denotes the use of any number of $\to_v$ steps, including zero steps. We also define $\to_v^n$ as $n$ consecutive $\to_v$ steps, with $n$ being a natural number. Note that $\to_v$ is deterministic, so there is only one way to apply $\to_v$ on a $\lambda$-term. The choice between using $\mu$, $\nu_<$ or no closure rule for $\to_v$ is deterministic. Consider the $\lambda$-term $MN$.

1. If $M$ is an abstraction and $N$ is a value, we can only apply $\beta_v$ directly.
2. If $M$ is an abstraction and $N$ is an application, we can only apply $\nu_<$ to reduce the left $\lambda$-term.
3. If $M$ is an application and $N$ any $\lambda$-term, we can only apply $\mu$ to reduce the left $\lambda$-term.

The closure rule $\nu_<$ forces evaluation from left to right, hence $<$ in the name. First, the closure rule $\mu$ is used after which $\nu_<$ is used. In Example 2.19, we give the cbv evaluation of $\lambda$-term $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$. Again, the $\beta$-redex that is reduced is underlined.

**Example 2.19. Cbv evaluation of $(\mathbf{I}(\lambda x.\mathbf{I}x))(\mathbf{II})$:**

$$\begin{array}{rcl} \underline{(\mathbf{I}(\lambda x.\mathbf{I}x))}(\mathbf{II}) & \to_v & (\lambda x.\mathbf{I}x)(\underline{\mathbf{II}}) \\ & \to_v & \underline{(\lambda x.\mathbf{I}x)\mathbf{I}} \\ & \to_v & \underline{\mathbf{II}} \\ & \to_v & \mathbf{I} \end{array}$$

# Chapter 3

# Call-by-box Lambda Calculus

This chapter introduces the call-by-box lambda calculus ($\lambda_b$), which is a refinement of another $\lambda$-calculus, $\lambda_\Box$ [15]. The calculus $\lambda_\Box$ is a fragment of Bierman and de Paiva's IS4 [8]. The fragment is so restricted that all typing rules are quite simple.

## 3.1 Specification of $\lambda_b$

### 3.1.1 Intuitionistic logic IS4

Intuitionistic modal logic IS4 is a non-classical logic that is based on the idea of constructive proof. Unlike classical logic, intuitionistic logic does not allow for the law of excluded middle, which states that for every proposition $P$, the statement $P \vee \neg P$ is assumed [20, 5]. In intuitionistic logic, a proposition is only considered true if it can be constructively proven. That is, a proposition is true if we can explicitly construct a proof, much like writing a program. This is why programming languages and proofs in systems like Agda and Rocq are based on intuitionistic logic. Since $\lambda$-calculus is the foundation of functional programming, it corresponds with intuitionistic logic via the Curry-Howard correspondence, where programs are equivalent to proofs and types to propositions [22].

The intuitionistic modal logic IS4 is an extension of intuitionistic logic that introduces the *modal operator* $\Box$. The term calculus $\lambda_\Box$ is a fragment of IS4 that, besides the usual $\lambda$-terms, also uses box (box) and unbox ($\varepsilon$) operators and is strong enough to interpret both call-by-name (cbn) and call-by-value (cbv) lambda calculi [15]. The box is like a container that delays a computation, making it possible to simulate both cbn and cbv evaluation strategies. Espírito Santo et al. refined this calculus to a simpler calculus, which they call the *call-by-box $\lambda$-calculus* ($\lambda_b$). This refinement has simpler

typing rules and only one $\beta$-reduction rule. In this thesis, we formalize $\lambda_b$ in Agda.

### 3.1.2 Terms and Types of $\lambda_b$

We now define the types of the $\lambda_b$ calculus as is done in [15]. We indicate the types of $\lambda_b$ with $A$ and $B$, representing types and boxed types respectively. The type $X$ denotes the base type. Note that function types are of the form $B \to A$ thus the input is always a boxed type.

**Definition 3.1. Types of $\lambda_b$** are given via the following grammar:

$$A \quad ::= \quad X \mid \ B \to A \mid \ B \qquad\qquad B \quad ::= \quad \Box A$$

We give some examples of types in $\lambda_b$. Note that $A$ can be any type that is not boxed, while $B$ can be any type that is boxed. The type $X$ is the base type.

**Example 3.2. Types in $\lambda_b$:**

$$\begin{aligned}
&X \\
&\Box(B \to A) \\
&\Box X \to X \\
&\Box A \to \Box B \\
&\Box\Box\Box X \to \Box X
\end{aligned}$$

Since variables are always boxed in $\lambda_b$, they always occur inside the unboxing operator $\varepsilon$. Thus, we only have $\varepsilon(x)$ in the grammar with $x$ a variable. Abstractions in $\lambda_b$, take boxed terms as arguments, which is why we define abstractions as $\lambda x.M$, where $x$ is the parameter of a boxed type and $M$ is a term. The rest of the terms are defined in Definition 3.3 and some example terms are given in Example 3.4. In this thesis, we use $M$, $N$, $P$, and $Q$ for arbitrary terms in $\lambda_b$.

**Definition 3.3. Terms in $\lambda_b$** are given by:

$$M, N, P, Q \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ MN \mid \ \mathsf{box}(N)$$

**Example 3.4. Terms in $\lambda_b$:**

$$\begin{aligned}
&\varepsilon(x) \\
&\lambda x.\varepsilon(x) \\
&(\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(y)) \\
&(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))
\end{aligned}$$

The typing rules of $\lambda_b$ are defined in Definition 3.5 and are identical to the typing rules in [15]. The $\Box$-I rule states that if $M$ is of type $A$, then $\mathsf{box}(M)$ is of type $\Box A$. The $\mathrm{Id}_\Box$ rule is the identity rule for boxed variables, which states that if we know that $x$ is a variable of type $\Box A$, we can conclude that $\varepsilon(x)$ is of type $A$. The only way to add variables to the context is by using the $\to$-I rule. These variables are always boxed since they are the parameters of abstractions. Due to the grammar of $\lambda_b$, all variables in $\lambda_b$ occur inside the unboxing operator $\varepsilon$ and thus the $\mathrm{Id}_\Box$ rule covers all cases. The $\to$-E rule states that if we have a term $M$ of type $B \to A$ and a term $N$ of type $B$, we can conclude that the application $MN$ has type $A$. It is important that the argument $N$ is a boxed term, which is why we denote its type as $B$.

**Definition 3.5. Typing rules of $\boldsymbol{\lambda_b}$** are defined as follows:

$$\frac{}{\Gamma, x : \Box A \vdash \varepsilon(x) : A}\ \mathrm{Id}_\Box$$

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.M : A \to B}\ \to\text{-I}$$

$$\frac{\Gamma \vdash M : B \to A \qquad \Gamma \vdash N : B}{\Gamma \vdash MN : B}\ \to\text{-E}$$

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \mathsf{box}(M) : \Box A}\ \Box\text{-I}$$

We now demonstrate these typing rules with Example 3.6, where we show that the term $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$ is well-typed. Note that the argument $\mathsf{box}(\lambda y.\varepsilon(y))$ is boxed. If the argument was not boxed, we would not be able to apply the abstraction $\lambda x.\varepsilon(x)$ to it.

**Example 3.6. Well-typedness of $\boldsymbol{(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))}$.** The typing derivation of $\lambda$-term $(\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y))$ is given by the tree below.

$$\dfrac{\dfrac{\dfrac{}{x : \Box(\Box A \to A) \vdash \varepsilon(x) : \Box A \to A}\ \mathrm{Id}_\Box}{\vdash \lambda x.\varepsilon(x) : \Box(\Box A \to A) \to (\Box A \to A)}\ \to\text{-I} \qquad \dfrac{\dfrac{\dfrac{}{y : \Box A \vdash \varepsilon(y) : A}\ \mathrm{Id}_\Box}{\vdash \lambda y.\varepsilon(y) : \Box A \to A}\ \to\text{-I}}{\vdash \mathsf{box}(\lambda y.\varepsilon(y)) : \Box(\Box A \to A)}\ \Box\text{-I}}{\vdash (\lambda x.\varepsilon(x))\mathsf{box}(\lambda y.\varepsilon(y)) : \Box A \to A}\ \to\text{-E}$$

## 3.2 Substitution in $\lambda_b$

Since variables always occur inside $\varepsilon$, we need to redefine free and bound variables and substitution. For $\lambda_b$, we have:

**Definition 3.7. Free and Bound Variables** in $\lambda_b$ are defined as follows:

$$\begin{array}{llll} FV_b(\varepsilon(x)) & = \{x\} & BV(\varepsilon(x)) & = \emptyset \\ FV_b(\lambda x.M) & = FV(M)\backslash\{x\} & BV(\lambda x.M) & = BV(M) \cup \{x\} \\ FV_b(MN) & = FV(M) \cup FV(N) & BV(MN) & = BV(M) \cup BV(N) \end{array}$$

**Definition 3.8. Substitution in** $\lambda_b$ of some $\lambda$-term $Q$ for an unboxed variable $\varepsilon(x)$ in some $\lambda$-term is defined by:

$$\begin{array}{ll} \varepsilon(x)[\varepsilon(x) \mapsto Q] & = Q \\ \varepsilon(y)[\varepsilon(x) \mapsto Q] & = \varepsilon(y) \text{ if } (x \neq y) \\ \mathsf{box}(M)[\varepsilon(x) \mapsto Q] & = \mathsf{box}(M[\varepsilon(x) \mapsto Q]) \\ (MN)[\varepsilon(x) \mapsto Q] & = M[\varepsilon(x) \mapsto Q]N[\varepsilon(x) \mapsto Q] \\ (\lambda x.M)[\varepsilon(x) \mapsto Q] & = \lambda x.M \\ (\lambda y.M)[\varepsilon(x) \mapsto Q] & = \lambda z.M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] \text{ if } (x \neq y) \end{array}$$

With $z$ a variable defined by:

1. If $x \notin FV_b(Q)$ or $y \notin FV_b(M)$ then $z = y$
2. Otherwise, $z$ can be any variable such that $z \notin FV(Q)$ or $z \notin FV(M)$

With this new $\lambda$-calculus, it may not be completely clear that substitution preserves well-typedness. We can prove that substitution preserves well-typedness in $\lambda_b$.

**Proposition 3.9. Preservation of well-typedness in $\lambda_b$:**

$$\text{If } \Gamma, x : \Box A \vdash M : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash M[\varepsilon(x) \mapsto Q] : A'$$

**Proof 3.10. Preservation of well-typedness in $\lambda_b$.** We prove by structural induction on the term $M$ that this substitution preserves well-typedness.

1. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash \varepsilon(x) : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash \varepsilon(x)[\varepsilon(x) \mapsto Q] : A'$$

   From the assumption, we have that $\Gamma, x : \Box A \vdash \varepsilon(x) : A'$. It follows that $x : \mathsf{box}(A)$ and thus $\varepsilon(x) : A$. We get that $A = A'$. Now, it follows that $\Gamma \vdash M[\varepsilon(x) \mapsto Q] : A'$ since $\varepsilon(x)[\varepsilon(x) \mapsto Q] = Q$.

2. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash \varepsilon(y) : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash \varepsilon(y)[\varepsilon(x) \mapsto Q] : A'$$

   We have that $\varepsilon(y)[\varepsilon(x) \mapsto Q] = \varepsilon(y)$, so $\Gamma \vdash \varepsilon(y)[\varepsilon(x) \mapsto Q] : A'$.

3. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash \mathsf{box}(M) : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash \mathsf{box}(M)[\varepsilon(x) \mapsto Q] : A'$$

   Let $M$ be of type $A''$ ($A' = \Box A''$). By the induction hypothesis, we get that $\Gamma \vdash M[\varepsilon(x) \mapsto Q] : A''$. It follows that $\mathsf{box}(M[\varepsilon(x) \mapsto Q]) : \Box A''$ or $\mathsf{box}(M[\varepsilon(x) \mapsto Q]) : A'$. Since $\mathsf{box}(M)[\varepsilon(x) \mapsto Q] = \mathsf{box}(M[\varepsilon(x) \mapsto Q])$, we get $\Gamma \vdash \mathsf{box}(M)[\varepsilon(x) \mapsto Q] : A'$

4. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash MN : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash MN[\varepsilon(x) \mapsto Q] : A'$$

   Let $N$ be of type $A''$. We get that $M$ has type $A'' \to A'$ because of the typing rule $\to$-E. By induction, we get:

   $$\Gamma \vdash M[\varepsilon(x) \mapsto Q] : A'' \to A' \text{ and } \Gamma \vdash N[\varepsilon(x) \mapsto Q] : A''$$

   Since $MN[\varepsilon(x) \mapsto Q] = M[\varepsilon(x) \mapsto Q]N[\varepsilon(x) \mapsto Q]$, and $M[\varepsilon(x) \mapsto Q]N[\varepsilon(x) \mapsto Q] : A'$, we get $\Gamma \vdash MN[\varepsilon(x) \mapsto Q] : A'$.

5. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash \lambda x.M : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash \lambda x.M[\varepsilon(x) \mapsto Q] : A'$$

   We have that $\lambda x.M[\varepsilon(x) \mapsto Q] = \lambda x.M$, since $x$ is a bound variable in $\lambda x.M$. It immediately follows that $\Gamma \vdash \lambda x.M[\varepsilon(x) \mapsto Q] : A'$.

6. We now prove:

   $$\text{If } \Gamma, x : \Box A \vdash \lambda y.M : A' \text{ and } \Gamma \vdash Q : A \text{ then } \Gamma \vdash \lambda y.M[\varepsilon(x) \mapsto Q] : A'$$

   Let $x \neq y$, $A' = B \to A''$, $y$ be of type $\Box B$ and $M$ be of type $A''$. By induction, we get the following with $z$ a fresh variable defined by the rules in Definition 3.8:

   $$\Gamma \vdash M[\varepsilon(y) \mapsto \varepsilon(z)] : A'' \text{ and } \Gamma \vdash M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] : A''$$

   From $\Gamma \vdash M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] : A''$ it follows that $\Gamma \vdash \lambda y.M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q] : B \to A''$, since $y$ is op type $B$. From this, the following statement follows:

   $$\lambda y.M[\varepsilon(x) \mapsto Q] = \lambda z.(M[\varepsilon(y) \mapsto \varepsilon(z)][\varepsilon(x) \mapsto Q])$$

   Now, we conclude $\Gamma \vdash \lambda y.M[\varepsilon(x) \mapsto Q] : A'$. $\Box$

With this definition of substitution, we define the reduction rule for $\lambda_b$ in Definition 3.11. In $\lambda_b$, the arguments of abstractions are always boxed, so we can only apply abstractions to boxed terms. Since variables only occur insides $\varepsilon$, we can replace those, including $\varepsilon$, with the corresponding unboxed term.

**Definition 3.11. $\beta$-reduction** in $\lambda_b$ is defined as follows:

$$(\lambda x.M)\mathsf{box}(N) \to M[\varepsilon(x) \mapsto N] \qquad (\beta_b)$$

For completeness, we need to define a new closure rule. This new rule only applies to the $\lambda_b$, and allows reduction under boxed terms. We call this rule $\zeta$, and Definition 3.12 defines it.

**Definition 3.12. The closure rule $\zeta$.** This rule allows reduction under boxed terms in $\lambda_b$.

$$\frac{M \to_{\beta_b} N}{\mathsf{box}(M) \to_{\beta_b} \mathsf{box}(N)}\ (\zeta)$$

The relations defined here are from [15] and form the reduction and evaluation relations of $\lambda_b$. The relation $\to_{\beta_b}$ allows reduction under all term formers, including boxes. We also define three other relations: $\beta_b$ closed under $\mu$ and $\nu$, $\beta_b$ closed under $\mu$ and $\beta_b$ closed under $\nu$. We call them $\to_{we}$, $\to_{b_n}$ and $\to_{b_v}$ respectively. Table 3.1 is an overview of all relations. The relation $\to_{we}$ is weak since values do not reduce. That is, abstractions and terms of the form $\varepsilon(x)$ do not reduce. The relation $\to_{we}$ is also external as boxes do not reduce. This relation is called call-by-box (cbb) evaluation. We will see after defining the embeddings that $\to_{we}$ collapses to $\to_{b_n}$ in Girard's embedding and that $\to_{we}$ collapses to $\to_{b_v}$ in Gödel's embedding.

| **Relation** | $\boldsymbol{\beta}$ | $\boldsymbol{\mu}$ | $\boldsymbol{\nu}$ | $\boldsymbol{\nu_<}$ | $\boldsymbol{\xi}$ | $\boldsymbol{\zeta}$ |
|---|---|---|---|---|---|---|
| $\rightarrow_{\beta_n}$ | $\beta_n$ | ✓ | ✓ | ✓ | ✓ | |
| $\rightarrow_n$ | $\beta_n$ | ✓ | | | | |
| $\rightarrow_w$ | $\beta_n$ | ✓ | ✓ | | | |
| $\rightarrow_{\beta_v}$ | $\beta_v$ | ✓ | ✓ | ✓ | ✓ | |
| $\rightarrow_v$ | $\beta_v$ | ✓ | | ✓ | | |
| $\rightarrow_{\beta_b}$ | $\beta_b$ | ✓ | ✓ | ✓ | ✓ | ✓ |
| $\rightarrow_{we}$ | $\beta_b$ | ✓ | ✓ | ✓ | | |
| $\rightarrow_{b_n}$ | $\beta_b$ | ✓ | | | | |
| $\rightarrow_{b_v}$ | $\beta_b$ | | ✓ | | | |

Table 3.1: Closure Rules for Different Reduction Relations

# Chapter 4

# Embeddings in $\lambda_b$

This chapter discusses the modal embeddings of the call-by-name $\lambda$-calculus ($\lambda_n$), and the call-by-value $\lambda$-calculus ($\lambda_v$) into the shared target, call-by-box $\lambda$-calculus ($\lambda_b$). We also give propositions about the embeddings, which are proven in Agda later in Section 5.3. These propositions state that reduction and evaluation are preserved by the embeddings, and have been proven several times in different contexts, for example, using linear logic or thunks [14, 24, 19]. However, $\lambda_b$ is a refined version of $\lambda_\Box$ [15]. We formalise these embeddings and prove their propositions in Agda.

## 4.1 Girard's embedding

In this section, we present Girard's embedding, which is a method to translate intuitionistic logic into linear logic [17]. Jean-Yves Girard introduced this embedding to represent intuitionistic logic in a linear framework, where resources are managed explicitly. Linear logic can be seen as a refinement of classical logic, where assumptions - or resources - are managed explicitly [31]. In particular, linear logic distinguishes between assumptions that should be used exactly once and those that can be used any number of times [31], including zero times. However, this thesis does not focus on linear logic, but on the way Girard's embedding can be used to represent the call-by-name $\lambda$-calculus in the modal call-by-box $\lambda$-calculus, $\lambda_b$.

In general, Girard's embedding is not limited to linear logic. It is a more general technique to represent one logical system in a different framework [32]. The embedding can be used to represent one logical system in another framework that has a way of managing resources or assumptions. In the context of $\lambda_n$, parameters of functions can be seen as resources that are consumed when the function is applied [24]. These resources can be used as many times as needed, hence the connection with Girard's embedding and linear logic.

The model embedding of the $\lambda_n$ into the $\lambda_b$ is given by Girard's trans-

lation, denoted as $(\cdot)^\circ$. Thus, $M^\circ$ is the translation of a term $M$ from the $\lambda_n$ to the $\lambda_b$, and $A^\circ$ is the translation of a type $A$ from the $\lambda_n$ to the $\lambda_b$. The translation is inductively defined in Definition 4.1 for both terms and types. We also give the translation of $\lambda$-term $\mathbf{I}x$ in Example 4.2.

**Definition 4.1. Girard's translation** $(\cdot)^\circ$ from $\lambda_n$ to $\lambda_b$ is defined as follows:

$$\begin{aligned} X^\circ &= X & x^\circ &= \varepsilon(x) \\ (A_1 \to A_2)^\circ &= \Box A_1{}^\circ \to A_2{}^\circ & (\lambda x.M)^\circ &= \lambda x.M^\circ \\ & & (MN)^\circ &= M^\circ \mathsf{box}(N^\circ) \end{aligned}$$

**Example 4.2. Girard's translation of $\mathbf{I}x$:**

$$\begin{aligned} (\mathbf{I}x)^\circ &= (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\ &= (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\ &= (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)). \end{aligned}$$

From Girard's translation, we see that a boxed $\lambda$-term can never occur on its own. Boxed terms are always part of an application, where the left-hand side is a term in $\lambda_b$ and the right-hand side is the boxed term. Therefore, the image of Girard's translation is a subset of the $\lambda_b$ given by the following grammar:

$$M, N \quad ::= \quad \varepsilon(x) \mid \ \lambda x.M \mid \ M\mathsf{box}(N)$$

We can reason that the relation $\to_{we}$ collapses to $\to_{b_n}$ when we consider the image of Girard's translation. Since application in Girard's image is always of the form $M\mathsf{box}(N)$, the right-hand side of the application can never be reduced without the use of the $\zeta$ rule. In $\to_{we}$, the only closure rules allowed are $\mu$ and $\nu$. The $\nu$ rule can never be applied in Girard's image as it is not possible to reduce under boxed terms. Thus, the only closure rule that can be applied is the $\mu$ rule, and $\to_{we}$ collapses to $\to_{b_n}$ in Girard's image.

We give a small example to clarify everything. Consider the $\lambda$-term $\mathbf{I}x$. It holds that $\mathbf{I}x \to_n x$ in $\lambda_n$. Example 4.3 shows that $(\mathbf{I}x)^\circ \to_{b_n} x^\circ$ also holds.

**Example 4.3. $(\mathbf{I}x)^\circ$ evaluated by $\to_{b_n}$:**

$$\begin{aligned} (\mathbf{I}x)^\circ &= (\lambda x.x)^\circ \mathsf{box}(x^\circ) \\ &= (\lambda x.x^\circ)\mathsf{box}(x^\circ) \\ &= (\lambda x.\varepsilon(x))\mathsf{box}(\varepsilon(x)) \\ &\to_{b_n} \varepsilon(x)[\varepsilon(x) \mapsto \varepsilon(x)] \\ &= \varepsilon(x) \\ &= x^\circ \end{aligned}$$

We now discuss the properties of Girard's translation. Girard's translation preserves the reduction and evaluation relations of $\lambda_n$. Preservation of reduction is the first proposition and is defined by Proposition 4.5. More specifically, it states that if a $\lambda$-term $M$ reduces to $N$ in $\lambda_n$, then $M^\circ$ reduces to $N^\circ$ in $\lambda_b$. The proof of this proposition is rather simple and is given in the chapter about the Agda formalisation, Section 5.3.3, though it requires a helper lemma, Lemma 4.4, which states that Girard's translation preserves and reflects substitution. The proof of this helper lemma is rather complex, because of the definition of substitution.

**Lemma 4.4. Preservation and reflection of substitution** under Girard's translation. For all $\lambda$-terms $M$, $N$ and $x$:

$$M^\circ[N^\circ \mapsto \varepsilon(x)] \equiv (M[N \mapsto x])^\circ$$

**Proposition 4.5. Preservation of reduction** under Girard's translation. For all $\lambda$-term $M$ and $N$:

$$\text{If } M \to_{\beta_n} N \text{ in } \lambda_n, \text{ then } M^\circ \to_{\beta_b} N^\circ.$$

The second proposition about Girard's translation is Proposition 4.6. It states that if a $\lambda$-term $M$ evaluates to $\lambda$-term $N$ with $\to_n$ in $\lambda_n$ in one step, then the translation of $M$ under Girard's embedding also evaluates to the translation of $N$ under Girard's embedding with $\to_{b_n}$ in one step. From this proposition, it easily follows that if $M$ evaluates to $N$ by call-by-name in $\lambda_n$, then $M^\circ$ evaluates by call-by-box to $N^\circ$ in $\lambda_b$.

**Proposition 4.6. Preservation of evaluation** under Girard's translation.

$$\text{For all } \lambda\text{-term } M \text{ and } N, \text{ if } M \to_n N \text{ in } \lambda_n, \text{ then } M^\circ \to_{b_n} N^\circ.$$

## 4.2 Gödel's Embedding

In this section, we present Gödel's embedding, introduced by Gödel in [18]. Gödel introduced this embedding as a method to translate intuitionistic logic into modal logic S4 [30, 18]. More specifically, an embedding to represent intuitionistic logic in a modal framework, where box modality, $\Box$ is used to indicate provability. That is, $\Box P$ can be read as $P$ is provable [30].

In general, Gödel's embedding is used to represent one more restricted logical system in a stronger framework [25]. This thesis uses Gödel's embedding to represent the call-by-name $\lambda$-calculus in the modal call-by-box $\lambda$-calculus, $\lambda_b$ [15]. The intuition behind Gödel's embedding is that boxes are used to represent values, which are already evaluated expressions [15].

In the model embedding of $\lambda_v$ into $\lambda_b$, we make use of an abbreviation, which we call raise similar to [15]. The idea of raise is to swap two $\lambda$-terms,

while removing a box of one $\lambda$-term. Its use becomes clear later in this section when we give some examples. We define $\mathsf{raise}$ as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The model embedding of $\lambda_v$ into $\lambda_b$ is given by Gödel's translation, denoted as $(\cdot)^*$. Thus, $M^*$ is the translation of a term $M$ from $\lambda_v$ to $\lambda_b$, and $A^*$ is the translation of a type $A$ from the $\lambda_v$ to the $\lambda_b$. We also define $(\cdot)^\bullet$ for the translation of values and types in $\lambda_v$. The embedding $(\cdot)^*$ and $(\cdot)^\bullet$ are mutually inductively defined in Definition 4.7 for both terms and types. That is, we define $(\cdot)^*$ in terms of $(\cdot)^\bullet$ and vice versa. We also give the translation of $\lambda$-term $\mathbf{I}x$ in Example 4.8.

**Definition 4.7** (h)**. Gödel's translation** $(\cdot)^*$ from $\lambda_v$ to $\lambda_b$ is defined as follows:

$$A^* = \Box A^\bullet \qquad\qquad V^* = \mathsf{box}(V^\bullet)$$
$$(MN)^* = \mathsf{raise}(N^*)M^*$$

$$X^\bullet = X \qquad\qquad x^\bullet = \varepsilon(x)$$
$$(A_1 \to A_2)^\bullet = \Box A_1{}^\bullet \to \Box A_2{}^\bullet \qquad (\lambda x.M)^\bullet = \lambda x.M^*$$

**Example 4.8. Gödel's translation of $\mathbf{I}x$:**

$$\begin{aligned}(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\ &= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\ &= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x)))\end{aligned}$$

Let us now reason about the image of Gödel's translation. From the translation, it follows that all applications in the image of Gödel's translation are of the form $\mathsf{raise}(M')M$, where both $M'$ and $M$ are terms in $\lambda_b$. Every term in the image that is not an application, is of the form $\mathsf{box}(V)$, with $V$ a value in $\lambda_b$. Thus, the image of Gödel's translation is a subset of $\lambda_b$ given by the following grammar:

$$M \quad ::= \quad \mathsf{box}(V) \mid \mathsf{raise}(M')M \qquad\qquad V \quad ::= \quad \varepsilon(x) \mid \lambda x.M$$

Now, we reason that the relation $\to_{we}$ collapses to $\to_{b_v}$ when we consider the image of Gödel's translation. We do this by showing that the only closure rule that can be applied to each type of term in Gödel's image is the $\nu$ rule.

1. If a term in Gödel's image is of the form $\mathsf{box}(V)$, we cannot apply any reduction rule, since we do not allow reductions under boxed terms. Besides, $V$ is a value, which cannot be reduced.

2. If a term in Gödel's image is of the form $\mathsf{raise}(M')M$, we can only apply the $\nu$ rule. We have that $\mathsf{raise}(M') = \lambda z.\varepsilon(z)M'$, which is a value. Thus, $\mathsf{raise}(M')$ cannot be reduced and the $\mu$ rule cannot be applied. The only closure rule that can be applied is the $\nu$ rule.

We give a small example to clarify everything. Consider the $\lambda$-term $\mathbf{I}x$. It holds that $\mathbf{I}x \to_v x$ in $\lambda_n$. Example 4.3 shows that $(\mathbf{I}x)^* \to_{b_v}^* x^*$ also holds.

**Example 4.9. $(\mathbf{I}x)^*$ evaluated by $\to_{b_v}$:**

$$\begin{aligned}
(\mathbf{I}x)^* &= \mathsf{raise}(x^*)(\lambda x.x)^* \\
&= \mathsf{raise}(\mathsf{box}(x^\bullet))\mathsf{box}((\lambda x.x)^\bullet) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.x^*) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(x^\bullet)) \\
&= \mathsf{raise}(\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&= (\lambda z.\varepsilon(z)\mathsf{box}(\varepsilon(x)))\mathsf{box}(\lambda x.\mathsf{box}(\varepsilon(x))) \\
&\to_{b_v} (\lambda x.\mathsf{box}(\varepsilon(x)))\mathsf{box}(\varepsilon(x)) \\
&\to_{b_v} \mathsf{box}(\varepsilon(x)) \\
&= \mathsf{box}(x^\bullet) \\
&= x^*
\end{aligned}$$

We now discuss the properties of Gödel's translation. Just like Girard's translation, Gödel's translation preserves reduction and evaluation. To prove these properties, we again need a helper lemma that is similar to the one used for Girard's translation. The helper lemma, Lemma 4.10, states that Gödel's translation preserves and reflects substitution. The lemma is divided into two parts - one concerning substitution within values, and the other dealing with substitution in any $\lambda$-term.

**Lemma 4.10. Preservation and reflection of substitution** under Gödel's translation. For all $\lambda$-terms $M$, values $V$, $W$, and variables $x$:

$$M^*[\varepsilon(x) \mapsto V^\bullet] \equiv (M[x \mapsto V])^* \text{ and } W^\bullet[\varepsilon(x) \mapsto V^\bullet] \equiv (W[x \mapsto V])^\bullet$$

Preservation of reduction of Gödel's translation is slightly more complicated than Girard's translation. This is because Gödel's translation makes use of the $\mathsf{raise}$ abbreviation, which swaps two terms. However, this swapping takes an extra step while reducing a $\lambda$-term. Thus, Proposition 4.11 states that if a $\lambda$-term $M$ in $\lambda_v$ reduces to $N$ in one step, then $M^*$ reduces to $N^*$ in $\lambda_b$ in two steps. We use the notation $\to_{\beta_b}^2$ for two-step reduction steps of $\to_{\beta_b}$.

**Proposition 4.11. Preservation of reduction** under Gödel's translation. For all $\lambda$-terms $M$ and $N$:

$$\text{If } M \to_{\beta_n} N \text{ in } \lambda_v, \text{ then } M^* \to_{\beta_b}^2 N^*.$$

The two steps that are taken next to each other are always similar and can be merged into one step. Let us reason about this.

Let us say that $\lambda$-term $P$ in $\lambda_v$ reduces to $\lambda$-term $Q$. We have that $P$ is of the form $(\lambda x.M)V$ and thus $Q$ is of the form $M[x \mapsto V]$. In Gödel's image, we have the following:

$$P^* \equiv \mathsf{raise}(V^*)(\lambda x.M)^* \equiv \mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x.M^*)$$

$$\begin{aligned}\mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x.M^*) &\equiv (\lambda z.\varepsilon(z)\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x.M^*)\\ &\to_{\beta_b} (\lambda x.M^*)\mathsf{box}(V^\bullet)\\ &\to_{\beta_b} M^*[\varepsilon(x) \mapsto V^\bullet]\end{aligned}$$

So, we have that $\mathsf{raise}(\mathsf{box}(V^\bullet))\mathsf{box}(\lambda x.M^*) \to_{\beta_b}^2 M^*[\varepsilon(x) \mapsto V^\bullet]$. More generally, Definition 4.12 defines the relation $\to_{\beta_{b2}}$.

**Definition 4.12. Gödel's two-step reduction relation $\to_{\beta_{b2}}$.**

$$\mathsf{raise}(\mathsf{box}(N))\mathsf{box}(\lambda x.M) \to_{\beta_{b2}} M[\varepsilon(x) \mapsto N] \qquad (\to_{\beta_{b2}})$$

With this new reduction relation, we can also formulate the preservation of reduction in one step. This is done in Proposition 4.13.

**Proposition 4.13. Preservation of reduction** under Gödel's translation with $\to_{\beta_{b2}}$ reduction rule. For all $\lambda$-terms $M$ and $N$:

$$\text{If } M \to_{\beta_v} N \text{ in } \lambda_v, \text{ then } M^* \to_{\beta_{b2}} N^*.$$

Preservation of evaluation is more difficult to prove than preservation of reduction. Proposition 4.6 states that if a $\lambda$-term $M$ evaluates to a value $V$ in $\lambda_v$ in any number of $\to_v$ steps, then $M^*$ evaluates to $V^*$ in $\lambda_b$ using any number of $\to_{b_v}$ steps. Alternatively, we can also say that if a $\lambda$-term evaluates by call-by-value to a value, then the translation of that $\lambda$-term under Gödel's embedding evaluates by call-by-box to the translation of that value under Gödel's embedding.

Due to time limitations, we did not prove Proposition 4.14 in the Agda formalization. However, a less formal proof of this proposition is given by José Espírito Santo, Luís Pinto, and Tarmo Uustalu in [15].

**Proposition 4.14. Preservation of evaluation** under Gödel's translation. For all $\lambda$-terms $M$ and values $V$:

$$\text{If } M \to_v V \text{ in } \lambda_v, \text{ then } M^* \to_{b_v} V^*.$$

# Chapter 5

# Formalisation in Agda

## 5.1 Intro to Agda

This section aims to provide a brief introduction to Agda, a dependently typed functional programming language and proof assistant. Agda uses strong typing and dependent types, and thus it can be used to write proofs constructively, which can be run as programs [27]. In order to understand the formalisation of $\lambda_b$ in Agda, we discuss the following basic concepts: data types, parameterised data types, functions, dependent data types, implicit arguments, and hole-based programming [2].

### 5.1.1 Data Types and Functions

Data types can be seen as a collection of values that have something in common. This collection is usually structured in some kind of way, such as a list or a tree [27]. Elements of a data type can be constructed using constructors, which are functions that return a value of the data type. In Agda, we can create a data type with the `data` keyword, followed by the name of the data type and its constructors [2].

Example 5.1 shows an example of a simple inductive data type: the natural numbers. It defines two constructors:

- `zero`, representing the base case (the number zero),
- `suc`, which takes a natural number and returns its successor.

The keyword `Set` in the type signature indicates the type of type. This is analogous to * in Haskell, which denotes the kind of types.

**Example 5.1. Natural numbers as a data type in Agda:**

```
data ℕ : Set where
  zero : ℕ
  suc : ℕ → ℕ
```

Data types in Agda can also have parameters. This makes them more flexible and allows for the creation of more complex data types. Example 5.2 shows such a data type, which is a list that can hold elements of any type $A$. We can make a list of natural numbers, which will have the type List $\mathbb{N}$, or a list of Booleans, which will have the type List Bool. Note that the parameter $A$ occurs before the colon, indicating that it is a type parameter.

**Example 5.2. List as a parameterised data type in Agda:**:

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A
```

Functions in Agda are defined using pattern matching, similar to Haskell. Each function has a type signature that defines the type of the function and its arguments. After the type signature, a number of equations follow, defining the function for each pattern, where functions can be recursive. Example 5.3 shows such a function, which doubles a natural number.

Functions can also be higher-order, taking other functions as arguments or returning functions as results. Function application of function $F$ to argument $x$ is written as $F\ x$ in Agda. Infix functions are denoted with underscores, such as _::_ in Example 5.2, where it is the constructor for the list data type. The first argument can be put on the left side of the constructor and the second argument on the right side, such as in $x :: xs$.

**Example 5.3. Function that doubles a natural number in Agda:**

```
double : ℕ → ℕ
double zero = zero
double (suc n) = suc (suc (double n))
```

Example 5.4 presents a dependent data type, which is a vector that has a length that is encoded in its type. In this example, we see that the type of a vector depends on a natural number, which is different for both constructors. The dependent type occurs after the colon, which is $\mathbb{N}$ in this case. Dependent types differ from parameterised types in that they can vary between constructors, whereas parameterised types use a single type parameter shared by all constructors.

The constructor _::_ in Example 5.4 is also an example of a dependent function. That is the return type of _::_ depends on the value of the implicit argument $n$. The argument $n$ is implicit, denoted by the curly braces around it.

We see that [] constructs a vector of length zero, resulting in type Vector $A$ zero, while the constructor _::_ constructs a vector of length $n + 1$, resulting in type Vector $A$ (suc $n$). Note that the implicit argument $n$ can be inferred from the input type Vector $A$ $n$.

**Example 5.4. Vector as a dependent data type in Agda:**

```
data Vector (A : Set) : ℕ → Set where
  [] : Vector A zero
  _::_ : {n : ℕ} → A → Vector A n → Vector A (suc n)
```

### 5.1.2 Hole-Based Programming

Agda is designed to be used with the Emacs text editor, and the combination provides a way to construct definitions and proofs interactively [1, 2, 27], which is called hole-based programming. While basic support for hole-based programming is available in other text editors, not all features are supported. Visual Studio Code, for example, has some basic support, while support for Vim is more limited and experimental [2].

In hole-based programming, terms can be constructed step by step, using question marks as placeholders. These turn into holes, indicating incomplete definitions. It is best to show this with an example. Note that we could use more holes in Example 5.5, but we feel like this example is sufficient for understanding the concept of hole-based programming.

**Example 5.5. Hole-based programming in Agda:**
This example defines the function plus that adds two natural numbers. It defines the function with a question mark, which we can fill in later. The type signature is already given.

```
plus : ℕ → ℕ → ℕ
plus = ?
```

Pressing the keys `C-c C-l` (while pressing the control key, typing c and then typing l) loads the file and transforms the question mark into a hole.

```
plus : ℕ → ℕ → ℕ
plus = {!   !}
```

We can now let Agda give the arguments of the plus function by pressing `C-c C-c`. The following prompt is shown:

```
pattern variables to case (empty for split on result):
```

Since we do not have any arguments yet, we can press enter.

```
plus : ℕ → ℕ → ℕ
plus x x₁ = {!   !}
```

Now, we would like to do a case distinction on the first argument. We let Agda do this by pressing `C-c C-c` again. The same prompt will be shown as before, but now we type the first argument $x$.

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = {!   !}
plus (suc x) x₁ = {!   !}
```

Inside the first hole, we press `C-c C-,`. This gives us information about the type of the hole and all the variables in scope, which are the variables that we can use to fill in the hole. We get something like the following output:

```
Goal: ℕ
--------------------------
x₁ : ℕ
```

This tells us we need to produce a natural number and that we can use the variable $x_1$ inside this hole. Since we have that $0 + x_1 = x_1$, we fill in the hole with $x_1$, which can be done by typing $x_1$ inside the hole and pressing `C-c C-space`. We can type the subscript 1 in Emacs by typing \ 1. Agda allows this usage for Unicode characters since Agda files are UTF-8 encoded [2]. The hole will be filled in and Agda will check if the type of the hole is correct. Pressing `C-c C-f` will move the cursor forward to the next hole.

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = {!   !}
```

Typing `C-c C-,` in this second hole gives us the type of the next hole. The output will be similar to the following:

```
Goal: ℕ
--------------------------
x₁ : ℕ
x : ℕ
```

The goal is to produce a natural number, which is the result of an addition. Since one of the arguments is `suc` $x$, we know the result is not zero, and thus we use the `suc` constructor. We fill in the hole with `suc ?` and type `C-c C-space`. We closed a hole and made a new hole, which is the argument of the `suc` constructor. We now have the following code:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc {!   !}
```

We can fill in the last hole with `plus` $x$ $x_1$ and press `C-c C-space` to close the hole. The final code looks like this:

```
plus : ℕ → ℕ → ℕ
plus zero x₁ = x₁
plus (suc x) x₁ = suc (plus x x₁)
```

Example 5.5 shows how to use the commands `C-c C-l`, `C-c C-c`, `C-c C-f`, `C-c C-space`, and `C-c C-,` to interactively develop a function in Agda. However, there are many more commands available in Agda to help with hole-based programming. For a complete list of commands, we refer to the Agda documentation about Emacs Mode [1].

## 5.2 Challenges of the Formalisation

This section discusses the challenges of the formalisation of $\lambda_b$ in Agda. The main challenge is dealing with variables and substitution. For variables, we used de Bruijn indices, which are a way to represent variables without names. The definition of substitution is based on the definition of substitution in the PLFA book [33], and is based on the definition by McBride [7]. The formalisation of $\lambda_b$ in Agda is restricted to well-typed terms, which avoids handling ill-typed terms. Another challenge is explicit weakening and renaming, which is needed to handle the contexts of $\lambda$-terms correctly. Two $\lambda$-terms with slightly different contexts are not the same, and we need to take this into consideration in the formalisation.

### 5.2.1 De Bruijn Indices

De Bruijn indices, introduced by De Bruijn, are a method to eliminate the need for variable names in the $\lambda$-calculus [9]. This is useful because now all $\lambda$-terms that are $\alpha$-equivalent are the same. We do not have to worry about variables with the same name that refer to different variables, or about variable capture, simplifying the formalisation in Agda. The definition of substitution becomes simpler because of this. This subsection first explains the concept of de Bruijn indices and then shows how they are implemented in the formalisation.

The $\lambda$-terms that use de Bruijn indices do not use variable names, but integers. We precede the integers with # to prevent confusion if the index is higher than 9. Without #, 10 could be confused with two indices 1 and 0 that occur next to each other. These integers represent the "distance" to the corresponding $\lambda$. Example 5.6 gives representations of $\lambda$-terms without free variables, using de Bruijn indices. These terms are said to be closed. Note that identical indices do not mean that the variables are the same. The $\lambda$-term $\lambda.\#0\lambda.\#0$ in Example 5.6 demonstrates this. In addition, variables that are the same can have different indices, which we demonstrate later.

**Example 5.6. De Bruijn indices for closed $\lambda$-terms:**

$$\begin{aligned}\lambda x.x &\equiv \lambda.\#0\\ \lambda x.\lambda y.x &\equiv \lambda.\lambda.\#1\\ \lambda x.\lambda y.y &\equiv \lambda.\lambda.\#0\\ \lambda x.x\lambda y.y &\equiv \lambda.\#0\lambda.\#0\end{aligned}$$

Representing open $\lambda$-terms ($\lambda$-terms with free variables) using de Bruijn indices is a little more complex. First, we represent the $\lambda$-term with de Bruijn indices, ignoring all free variables. The idea here is that we wrap the entire $\lambda$-term inside an imaginary $\lambda$ that binds the first free variable, and we give it an index the same way as we do for bound variables. We keep adding these imaginary $\lambda$s until we have no more free variables left if we consider this wrapped $\lambda$-term. Example 5.7 illustrates this, with the final $\lambda$-term being more complex. It is the same example that de Bruijn presented in his paper [9].

The last term in Example 5.7 may need some explanation. We first wrap the $\lambda$-term with three imaginary $\lambda$s, corresponding to $z$, $w$ and $y$ respectively. The $\lambda$ that binds $z$ is the outermost $\lambda$. The indices #2 and #3 both refer to $w$, because index 3 is one $\lambda$ deeper, and the de Bruijn indices can be seen as the distance between the corresponding $\lambda$ and the variable.

**Example 5.7. De Bruijn indices for open $\lambda$-terms:**

$$\begin{aligned}x\lambda y.y \equiv x\lambda.\#0 &\equiv \#0\lambda.\#0\\ x\lambda y.x \equiv x\lambda.x &\equiv \#0\lambda.\#1\end{aligned}$$

$$\begin{aligned}\lambda x.(\lambda t.(xt(\lambda u.utz)(\lambda s.w))wy) &\equiv \lambda.(\lambda.(\#1\#0(\lambda.\#0\#1z)(\lambda.w))wy)\\ &\equiv \lambda.(\lambda.(\#1\#0(\lambda.\#0\#1\#5)(\lambda.\#3))\#2\#1)\end{aligned}$$

In the formalisation, we will represent de Bruijn indices similar to how natural numbers are represented in Agda. De Bruijn index 0 is represented as `Z`, de Bruijn index 1 as `S Z`, de Bruijn index 2 as `S (S Z)`, and so on.

### 5.2.2 Well-typed Terms

This subsection discusses the restriction to well-typed terms in the formalisation of $\lambda_b$. The restriction to well-typed terms is important because it avoids all complications that arise from ill-typed terms. In particular, in the context of evaluation, it makes sense to restrict to well-typed terms, because ill-typed terms do not have a normal form. Evaluation is only defined for well-typed terms, and thus the restriction to well-typed terms makes sense.

The formalisation uses an intrinsically typed approach where terms are defined together with their types. In the formalisation, $\lambda$-terms are automatically associated with a type, and we can only construct well-typed terms.

This approach is also referred to as Church style typing, and was introduced by Alonzo Church in [11].

The formalisation is based on the representation proposed by AltenKirch and Reus in [3]. We briefly go over the types and $\lambda$-terms in the formalisation, and how they are defined. Definition 5.8 gives the formal definition of types in the $\lambda$-calculus.

**Definition 5.8. Types** in the $\lambda$-calculus:

```
data Type : Set where
  _⇒_ : Type → Type → Type
  X : Type
```

Terms in the $\lambda$-calculus are defined together with a context, which can be seen as a list of Types. We do not need to name the variables in the context, because we use de Bruijn indices to refer to them. The context is a list of types, where each type corresponds to a variable in the context. Definition 5.9 gives the formal definition of contexts in the $\lambda$-calculus. This construction looks somewhat similar to the list data type that we gave as an example in Example 5.2.

**Definition 5.9. Context** in the $\lambda$-calculus:

```
data Context : Set where
  ∅ : Context
  _,_ : Context → Type → Context
```

We now define de Bruijn indices that refer to the variables in the context. They can be seen as proof that some type is in the context. If $A$ occurs in the context, then $A \in \Gamma$ denotes that $A$ is in the context $\Gamma$, and de Bruijn index is the proof of this. It gives the location in the context where the type occurs. The de Bruijn index Z refers to the most recently introduced variable in the context, which appears on the rightmost position of the context. The de Bruijn index S Z refers to the second most recently introduced variable, and so on.

**Definition 5.10. De Bruijn indices** in the $\lambda$-calculus:

```
data _∈_ : Type → Context → Set where
  Z : ∀ {Γ A} → A ∈ (Γ , A)
  S : ∀ {Γ A B} → A ∈ Γ → A ∈ (Γ , B)
```

Definition 5.11 gives the formal definition of terms in the $\lambda$-calculus. The $\lambda$-terms are defined by the context they are in and the type they have. For example, $\Gamma \vdash A$ denotes the $\lambda$-terms that have type $A$ in the context $\Gamma$.

With the constructors ', $\lambda$ and $\cdot$, we can construct $\lambda$-terms. We cannot use the $\lambda$ for $\lambda$ because it is already used in Agda for making functions. The ' constructor is used to construct $\lambda$-terms that are variables if we can provide evidence that the variable is in the context. Abstractions are constructed with the $\lambda$ constructor, which takes a context with more than one variable and creates a new $\lambda$-term that has one less variable in the context. This new $\lambda$-term is the abstraction of the original $\lambda$-term. The $\cdot$ constructor is used to construct applications of $\lambda$-terms. Given a $\lambda$-term that has a function type and an argument of the input type of the function type, $\cdot$ constructs a new $\lambda$-term of the return type of the function type.

Examples of $\lambda$-terms are given in Example 5.12. Note that we need to give the type of the $\lambda$-term, which is of the form $\Gamma \vdash A$, and that we provide evidence for this type when constructing the $\lambda$-term.

**Definition 5.11. Data type of $\lambda$-terms in Agda:**

```
data _⊢_ : Context → Type → Set where
  ‘ : ∀ {Γ A}
    → A ∈ Γ
      -------
    → Γ ⊢ A

  ƛ : ∀ {Γ A B}
    → Γ , A ⊢ B
      --------------
    → Γ ⊢ A ⇒ B

  _·_ : ∀ {Γ A B}
    → Γ ⊢ A ⇒ B
    → Γ ⊢ A
      --------------
    → Γ ⊢ B
```

**Example 5.12. $\lambda$-terms in Agda.** In this example, we show how to construct $\lambda$-terms in Agda. We also show in comments what $\lambda$-terms they correspond to if they are written in the traditional style.

```
t₁ : ∅ , X ⊢ X
t₁ = ‘ Z
-- same as x

t₂ : ∅ ⊢ X ⇒ X
t₂ = ƛ (‘ Z)
-- same as λx.x
```

```
t₃ : ∅ , X ⊢ X
t₃ = ƛ (` Z) · ` Z
-- same as (λx.x)y

t₄ : ∅ , X ⇒ X ⊢ X ⇒ X
t₄ = ƛ (` (S Z) · (` (S Z) · ` Z))
-- same as λx.y(yx)

t₅ : ∅ , X ⇒ X , X ⊢ X ⇒ X
t₅ = ƛ (` (S (S Z)) · (` (S (S Z)) · ` Z))
-- same as λx.y(yx), but with a different context
```

Now that we have defined types, contexts and terms in the standard $\lambda$-calculus, we define the $\lambda$-terms in the $\lambda_b$. Definition 5.13 gives the formal definition of $\lambda$-terms in the $\lambda_b$. Note that b is the suffix of many constructors, which is to distinguish between the constructors of the standard $\lambda$-calculus and $\lambda_b$.

**Definition 5.13. $\lambda_b$ in Agda.** This definition gives the data types for types, contexts and $\lambda$-terms of $\lambda_b$ in Agda. BOX A is a convenient abbreviation for Box (□ A). The definition of the $\lambda$-terms is based on the typing rules of $\lambda_b$, which are given in Definition 2.3.

```
data Typeb : Set
data BoxedTypeb : Set

data Typeb where
  _⇒b_ : BoxedTypeb → Typeb → Typeb
  X : Typeb
  Box : BoxedTypeb → Typeb

data BoxedTypeb where
  □ : Typeb → BoxedTypeb

BOX : Typeb → Typeb
BOX A = Box (□ A)

data Contextb : Set where
  ∅b : Contextb
  _,b_ : Contextb → BoxedTypeb → Contextb

data _∈b_ : BoxedTypeb → Contextb → Set where
  Zb : ∀ {Γ □A} → □A ∈b (Γ ,b □A)
  Sb : ∀ {Γ □A □B} → □A ∈b Γ → □A ∈b (Γ ,b □B)

data _⊢b_ : Contextb → Typeb → Set where
```

```
ε : ∀ {Γ A}
  → □ A ∈b Γ
    -------
  → Γ ⊢b A

box : ∀ {Γ A}
  → Γ ⊢b A
  -------------
  → Γ ⊢b BOX A

ƛb : ∀ {Γ A B}
  → Γ ,b □ A ⊢b B
    --------------
  → Γ ⊢b □ A ⇒b B

_·b_ : ∀ {Γ A B}
  → Γ ⊢b □ A ⇒b B
  → Γ ⊢b BOX A
    --------------
  → Γ ⊢b B
```

### 5.2.3 Substitution

The next challenge in the formalisation is a correct definition of substitution. This definition appeared to be the most complex part of the formalisation, and it caused the most problems with the proofs later in this thesis. The definition of substitution is similar to the definition that is used in the chapter "De Bruijn indices" in the PLFA book [33]. It is based on the definition by McBride [7], which gives a similar definition of substitution in Rocq.

We only give the definition of substitution in Agda of $\lambda$-terms in $\lambda_b$, since the definition of substitution in the standard $\lambda$-calculus is exactly the same as the definition in [33]. Definition 5.15 gives the definition of parallel substitution, which is the substitution of a list of variables by a list of $\lambda$-terms. Then we define single substitution in Definition 5.16, which might be more familiar and is the substitution of a single $\lambda$-term by another $\lambda$-term.

In general, substitution is about replacing variables by $\lambda$-terms. With parallel substitution, multiple variables are replaced by other $\lambda$-terms. So, substitution can be seen as a function, taking a map from variables in one context to $\lambda$-terms in another context and a $\lambda$-term as arguments, and returning a new $\lambda$-term with the variables replaced by the corresponding $\lambda$-terms.

For the definition of parallel substitution, we need to have an extension lemma, which is needed when we encounter an abstraction. An abstraction adds a new variable to the context, and we need a way to use the substitution

function in the new context. This is where the extension lemma extsb comes in. It is a function from a map from variables in one context to $\lambda$-terms in another context, to a map from variables in the first context extended with a type to $\lambda$-terms in the second context extended with a type. The type that is used for extending the context is the input type of the $\lambda$ abstraction. We can think of extsb as a way to apply some mapping function to the body of a $\lambda$ abstraction while keeping the parameter of the abstraction untouched.

The extension lemma is defined in terms of renameb, which is used to construct a $\lambda$-term with an extended context. We can think of renameb as a function that "renames" variables. We can change the context and adjust the de Bruijn indices accordingly. In de case of substitution, we "rename" the $\lambda$-term by adding the input parameter of the $\lambda$ abstraction to the context and by increasing all de Bruijn indices by one. The renameb function takes a function from variables in one context to variables in another context, and returns a function from $\lambda$-terms in the first context to $\lambda$-terms in the second context. For example, renameb Sb $M$ renames $M$ by applying the function Sb to the variables in $M$. An example of this is given in Example 5.14.

The rename lemma is defined in terms of extb, which works similarly to extsb, but is only used for extending the context. We can think of extb as a way to extend the context. Given a map from one context to another, it extends the map by adding some $\lambda$-term to the context. The function extb takes a map from variables in one context to variables in another context and returns a map from variables in the first context extended with a type to variables in the second context extended with a type. Again, this type will be the input type of the $\lambda$ abstraction in renameb. Thus, the definition of simultaneous substitution is based on the definitions of extsb, renameb and extb.

**Example 5.14. Renaming in Agda.** Renaming $\mathsf{t}_1$ with Sb results in $\mathsf{t}_2$.

```
t₁ : ∅b ,b □ (□ X ⇒b X) ⊢b □ X ⇒b X
t₁ = ƛb (ε (Sb Zb) ·b box (ε (Sb Zb) ·b box (ε Zb)))

t₂ : ∅b ,b □ (□ X ⇒b X) ,b □ X ⊢b □ X ⇒b X
t₂ = ƛb (ε (Sb (Sb Zb)) ·b box (ε (Sb (Sb Zb)) ·b box (ε Zb)))


_ : renameb Sb t₁ ≡ t₂
_ = refl
```

**Definition 5.15. Parallel substitution in Agda of $\lambda_b$.** We define Parallel substitution in terms of extsb, renameb and extb.

```
extb : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
```

```
      ----------------------
  → (∀ {A B} → A ∈b Γ ,b B → A ∈b Δ ,b B)
extb f Zb = Zb
extb f (Sb x) = Sb (f x)

rename : ∀ {Γ Δ}
  → (∀ {A} → A ∈b Γ → A ∈b Δ)
      ----------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
rename f (ε x) = ε (f x)
rename f (box N) = box (rename f N)
rename f (ƛb M) = ƛb (rename (extb f) M)
rename f (M ·b N) = rename f M ·b (rename f N)

exts : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
      ----------------------
  → (∀ {A B} → □ A ∈b Γ ,b B → Δ ,b B ⊢b A)
exts f Zb = ε Zb
exts f (Sb x) = rename Sb ( f x )

substb : ∀ {Γ Δ}
  → (∀ {A} → □ A ∈b Γ → Δ ⊢b A)
      ------------------------
  → (∀ {A} → Γ ⊢b A → Δ ⊢b A)
substb f (ε x) = f x
substb f (box N) = box (substb f N)
substb f (ƛb M) = ƛb (substb (exts f) M)
substb f (M ·b N) = substb f M ·b (substb f N)
```

Single substitution can be defined by using parallel substitution. The map fb in Definition 5.16 is a function that takes a variable in the context and returns a $\lambda$-term. It only returns the $\lambda$-term that is substituted if the variable is the most recent variable in the context. Single substitution of the $\lambda$-term that occurs most recently in the context of $M$ by $\lambda$-term $N$ is denoted by $M$ [ $N$ ]b in the formalisation of $\lambda_b$.

**Definition 5.16. Single substitution in Agda of $\lambda_b$.** Single substitution is defined in terms of parallel substitution, using the function fb that replaces the most recent variable in the context with the $\lambda$-term that is substituted.

```
fb : ∀ {Γ C} (M : Γ ⊢b C) → ∀ {A} → □ A ∈b Γ ,b □ C → Γ ⊢b A
fb M Zb = M
fb M (Sb x) = ε x
```

```
_[_]b : ∀ {Γ A C}
  → Γ ,b □ C ⊢b A
  → Γ ⊢b C
    --------------
  → Γ ⊢b A
_[_]b N M = substb (fb M) N
```

Example 5.17 gives a demonstration of single substitution in $\lambda_b$.

**Example 5.17. Single substitution in Agda of $\lambda_b$.** This example shows that $t_1$ [ $t_2$ ]b ≡ $t_3$

```
t₁ : ∅b ,b □ (□ X ⇒b X) ⊢b □ X ⇒b X
t₁ = ƛb (ε (Sb Zb) ·b box (ε (Sb Zb) ·b box (ε Zb)))

t₂ : ∅b ⊢b □ X ⇒b X
t₂ = ƛb (ε Zb)

t₃ : ∅b ⊢b □ X ⇒b X
t₃ = ƛb (ƛb (ε Zb) ·b box (ƛb (ε Zb) ·b box (ε Zb)))

_ : t₁ [ t₂ ]b ≡ t₃
_ = refl
```

### 5.2.4 Weakening and Renaming

Weakening and renaming becomes something to take into consideration in the formalisation. Two $\lambda$-terms that are the same except for the context are not exactly the same. An example to demonstrate this is the formal definition of raise in Agda, where the use of renameb is required. Recall that raise was defined as follows:

$$\mathsf{raise}(M) := \lambda z.\varepsilon(z)M$$

The term $M$ outside the function (in $\mathsf{raise}(M)$) has a different context than the term $M$ inside the function of raise ($\lambda z.\varepsilon(z)M$). We attempt to give a typing derivation in Figure 5.1. We assume that term $M$ has type B' which can be derived from $\Gamma$. However, we are stuck in the derivation since we do not have rules to discard assumptions. We see that we need to prove $\Gamma, z : \Box(\Box B \to B') \vdash M : B'$ instead of $\Gamma \vdash M : B'$.

$$\dfrac{\dfrac{}{\Gamma, z : \Box(\Box B \to B') \vdash \varepsilon(z) : \Box B \to B'}\,\mathrm{Id}_{\Box} \qquad \begin{array}{c}\vdots \\ \Gamma, z : \Box(\Box B \to B') \vdash M : B'\end{array}}{\dfrac{\Gamma, z : \Box(\Box B \to B') \vdash \varepsilon(z) M : B'}{\Gamma \vdash \lambda z.\varepsilon(z) M : \Box(\Box B \to B') \to B'}\,{\to}\text{-I}}\,{\to}\text{-E}$$

Figure 5.1: Attempt of the typing derivation of $\lambda z.\varepsilon(z)M$

Though we cannot immediately conclude the statement below, it does make sense that this is the case.

$$\Gamma \vdash M : B' \text{ implies } \Gamma, z : \Box(\Box B \to \Box B') \vdash M : \Box B'$$

We only add an extra assumption that we do not use, so we need a function that can handle this. This is where renameb comes in. The function renameb "renames" the de Bruijn indices in the $\lambda$-terms with a certain function and changes the context accordingly. In our case, we only need to rename the indices with Sb (S for $\lambda_b$).

Let us show how this works in the formalisation. Assume that we have a $\lambda$-term $M$ that has type $B'$ in context $\Gamma$, which is denoted by $M : \Gamma \vdash B'$ in the formalisation. We use renameb in such a way that it returns a $\lambda$-term of the type $\Gamma$ , b $\Box(\Box B \to B') \vdash \Box B'$. We achieve this by renaming with the function Sb as shown below. The resulting type is $\Gamma$ , b $\Box(\Box B \to \Box B') \vdash \Box B'$

$$\mathsf{rename}\ (\mathsf{Sb}\ \{\Box B = \Box(\Box B \to \Box B')\})\ M$$

Here, we need to give the implicit argument since Sb needs to know what the resulting context will be. Now, we incorporate this into the definition of raise. Note that Agda can automatically infer the implicit arguments of Sb in the definition of raise.

**Definition 5.18.** **raise** in Agda:

```
raise : ∀ {Γ A B}
  → (t : Γ ⊢b BOX B)
  → Γ ⊢b □ (□ B ⇒b BOX A) ⇒b BOX A
raise t = ƛb (ε Zb ·b renameb Sb t)
```

## 5.3 Overview of the Formalisation

This section provides an overview of the formalisation in Agda. We show how propositions in Section 4.1 and Section 4.2 are formalised in Agda. We

also give the main helper lemmas that are needed to prove these propositions. The formalisation is available on GitHub[1]

### 5.3.1 File Structure

The Agda formalisation is structured into a number of files. Here, we go over the file structure. The project has the following three main directories: `examples`, `formalisation` and `proofs`. The `examples` directory contains some examples of $\lambda$-terms, substitution, embeddings and reductions. The `formalisation` directory is the most important directory and contains the formalisation of $\lambda$-terms, substitution, embeddings and reductions. Here, we list all files in this directory and give a short description of their contents:

- `boxCalc.agda`: Contains the definition of $\lambda$-terms in $\lambda_b$, the definition of substitution and the definition of values in $\lambda_b$.
- `boxCalcRed.agda`: Contains all the reduction relations in $\lambda_b$.
- `calc.agda`: Contains the definition of $\lambda$-terms in the standard $\lambda$-calculus, the definition of substitution and the definition of values.
- `embedCBNIntoCBB.agda`: Contains the definition of Girard's embedding.
- `embedCBVIntoCBB.agda`: Contains the definition of Gödel's embedding.
- `nameCalcRed.agda`: Contains the reduction relations for $\lambda_n$.
- `valueCalcRed.agda`: Contains the reduction relations for $\lambda_v$.

The `proofs` directory contains the proofs of the propositions in Section 4.1 and Section 4.2 in the files `nameProofs.agda` and `valueProofs.agda` respectively. Both files also contain all helper lemmas necessary to prove the propositions.

### 5.3.2 Helper Lemmas

This subsection goes over the two main helper lemmas needed to prove the main propositions about Girard's and Gödel's embedding. In general, the lemma pres-subst states that the order of embedding and substitution does not matter. Lemma 5.19 shows the corresponding Agda statement of this lemma for Girard's embedding. We do not show the proof to make this thesis more readable, but it can be found in the Agda formalisation in `proofs/nameProofs.agda`. Besides, the full proof is quite long and involves

[1] `https://github.com/rodepanda7/agda-call-by-box`

going through the chained definition of substitution, traversing extsb, renameb and extb. Lemma 5.20 shows the statement of pres-subst for Gödel's embedding in Agda. This lemma consists of two parts, one for substitution in values and one for substitution in $\lambda$-terms in general. The complete proof of this can be found in the Agda formalisation in `proofs/valueProofs.agda`.

**Lemma 5.19. Preservation of single substitution in Girard's embedding.** This lemma is defined in `proofs/nameProofs.agda`

```
-- Preservation and Reflection of
-- single substitution of Girard's translation
pres-subst : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (N : Γ ⊢ B)
  → (embedTerm M) [ embedTerm N ]b ≡ embedTerm (M [ N ])
```

**Lemma 5.20. Preservation of single substitution in Gödel's embedding.** This lemma is defined in `proofs/valueProofs.agda`

```
-- Preservation and Reflection of
-- single substitution of Gödel's translation
pres-subst : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (V : Γ ⊢ B)
  → (x : Value V)
  → embedTerm M [ embedValue V x ]b ≡ embedTerm (M [ V ])


-- Preservation and Reflection of
-- single substitution in values of Gödel's translation
pres-subst-val : ∀ {Γ A B}
  → (M : Γ , B ⊢ A)
  → (V : Γ ⊢ B)
  → (x : Value M)
  → (y : Value V)
  → embedValue M x [ embedValue V y ]b
    ≡ embedValue (M [ V ]) (val-after-subst M V x y)
```

Due to the use of renameb in the definition of raise in Gödel's embedding, we also needed many other helper lemmas such as subst-renameb-comp, substb-rename-commute and substb-cong. Since subst is defined by extsb, which is defined by renameb and extb, we also need composition, commutativity and congruence lemmas for extsb, renameb and extb. However, these lemmas are not that interesting for this thesis, and we do not give them here. These lemmas can be found in the Agda formalisation in `proofs/valueProofs.agda`.

### 5.3.3 Proofs of the Main Propositions

This subsection gives proofs of the propositions about Girard's and Gödel's embedding. We start with the propositions about Girard's embedding, and then we continue with the propositions about Gödel's embedding. The first proposition, Proposition 4.5, states that Girard's embedding preserves reduction. That is, if we have a $\lambda$-term $M$ and we reduce it to $N$ in $\lambda_n$, then the translation of $M$ under Girard's embedding also reduces to the translation of $N$ under Girard's embedding. We repeat the proposition here for convenience:

For all $\lambda$-term $M$ and $N$, if $M \rightarrow_{\beta_n} N$ in $\lambda_n$, then $M^\circ \rightarrow_{\beta_b} N^\circ$.

Proof 5.21 shows the Agda proof of this proposition. Note that we use the helper Lemma 4.4 to prove this proposition.

**Proof 5.21. Preservation of reduction under Girard's translation.**

```
-- Preservation of reduction of Girard's translation
pres-red : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βn N)
  → (embedTerm M ⇝βb embedTerm N)
```

□

The second proposition about Girard's translation is Proposition 4.6. It states that if a $\lambda$-term $M$ evaluates to $N$ in $\lambda_n$ with the relation $\rightarrow_n$, then the translation of $M$ under Girard's embedding also evaluates to the translation of $N$ under Girard's embedding with $\rightarrow_{b_n}$. Again, we repeat the proposition here for convenience:

For all $\lambda$-term $M$ and $N$, if $M \rightarrow_n N$ in $\lambda_n$, then $M^\circ \rightarrow_{b_n} N^\circ$.

Proof 5.22 shows the Agda proof of this proposition. The proof is similar to proof 5.21. The only difference is that we have fewer cases for the evaluation relation $\rightarrow_n$ and thus the proof is shorter.

**Proof 5.22. Preservation of evaluation under Girard's translation.**

```
-- Preservation of evaluation of Girard's translation
pres-eval : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝n N)
  → (embedTerm M ⇝bn embedTerm N)
```

□

The next two propositions are about Gödel's embedding. The first proposition of Gödel's embedding we proved is Proposition 4.11. It states if a $\lambda$-term $M$ in $\lambda_v$ reduces to $N$ in one step, then $M^*$ reduces to $N^*$ in $\lambda_b$ in two steps. We repeat the proposition here for convenience:

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_n} N$ in $\lambda_v$, then $M^* \to^2_{\beta_b} N^*$.

Proof 5.24 shows the Agda proof of this proposition. Note that it makes use of the helper lemma pres-rename-red-2, which states that if a $\lambda$-term $M$ reduces to $N$ in two steps, then renameb $fb'$ $M$ also reduces to renameb $fb'$ $N$ in two steps. We do not go into detail about this lemma, but it can be found in `proofs/valueProofs.agda`.

**Proof 5.23. Preservation of reduction** under Gödel's translation with $\to^2_{\beta_b}$ reduction rule.

```
-- Preservation of 2 step reduction of Gödel's translation
pres-red : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βv N)
  → (embedTerm M ⇝βb² embedTerm N)
```

□

The last proposition we proved is Proposition 4.11. It states that if $M \to_{\beta_v} N$ in $\lambda_v$, then $M^* \to_{\beta_{b2}} N^*$. We repeat the proposition here for convenience:

For all $\lambda$-terms $M$ and $N$, if $M \to_{\beta_v} N$ in $\lambda_n$, then $M^* \to_{\beta_{b2}} N^*$.

The proof of this proposition is similar to proof 5.23 and is given in proof 5.24. We use the helper lemma pres-rename-red- $\rightsquigarrow \beta_{b2}$ that is similar to pres-rename-red-2 but for the $\to_{\beta_{b2}}$ reduction rule. The proof of this lemma can be found in `proofs/valueProofs.agda`.

**Proof 5.24. Preservation of reduction** under Gödel's translation with $\to_{\beta_{b2}}$ reduction rule.

```
-- Preservation of reduction by Gödel's translation
-- with the two step reduction rule
pres-red⇝βb₂ : ∀ {Γ A}
  → (M : Γ ⊢ A)
  → (N : Γ ⊢ A)
  → (M ⇝βv N)
  → (embedTerm M ⇝βb₂ embedTerm N)
```

□

# Chapter 6

# Related Work

Several approaches have been proposed to unify call-by-name and call-by-value evaluation strategies. This section goes over the most notable ones.

Levy's approach [23] introduces a programming paradigm called call-by-push-value (CBPV), which is based on the quote "a value is, a computation does". CBPV distinguishes between values and computations. It can be constricted to simulate call-by-name in one way and call-by-value in another. An extensive formalisation of CBPV in Rocq is provided by Forster et al. in [16].

Another approach to unifying cbn and cbv is by the use of linear logic. Maraist et al. [24] describe how two translations by Girard from intuitionistic logic into linear logic can be used to simulate call-by-name, call-by-value, and call-by-need evaluation strategies. The idea is that linear logic can be used to keep track of the usage and evaluation of resources, which in this case are the arguments of a function. Ehrhard and Guerrieri [14] also used linear logic to unify cbn and cbv. They introduced the Bang Calculus, which is a linear logic-based $\lambda$-calculus that can express both cbn and cbv by using Girard's translations. A formalisation of a linear logic-based $\lambda$-calculus in Rocq is provided by Zhao, Zhang, and Zdancewic in [35], with the implementation on GitHub [34].

The next approach we discuss uses thunks to unify cbn and cbv. Thunks can be seen as wrappers around an expression that delays the evaluation until it is needed. Continuous-Passing Style (CPS) translations are also a technique to unify cbn and cbv. CPS translations are a way to rewrite a program such that all functions take an extra argument, which is the continuation function. This continuation function is called with the result of the function, and it indicates the next step in the computation. Hatcliff and Danvy [19] used thunks and CPS translations to unify cbn and cbv. There is one formalisation of a CPS translation in Rocq by Torrens [29], though it is not a direct translation of the approach by Hatcliff and Danvy.

The final approach uses model logic to unify cbn and cbv, and this is the

approach we formalised in Agda. Espírito Santo et al. [15] introduced the modal $\lambda$-calculus $\lambda_{\Box}$, which is a fragment of Bierman and de Paiva's intuitionistic modal logic (IS4) [8]. The modal calculus $\lambda_{\Box}$ can express both cbn and cbv by using Girard's and Gödel's embeddings, respectively. Espírito Santo et al. refined $\lambda_{\Box}$ to a simpler $\lambda$-calculus, which they called the call-by-box $\lambda$-calculus ($\lambda_b$). This refinement has simpler typing rules and a more straightforward reduction strategy. No formalisation of $\lambda_b$ has been made in a proof assistant yet, and thus we formalised $\lambda_b$ in Agda.

# Chapter 7

# Conclusions and Future Work

## 7.1 Conclusions

In this thesis, we have formalised the call-by-box lambda calculus, a refined modal $\lambda$-calculus introduced by [15]. We implemented the syntax, substitution, reduction and evaluation relations of $\lambda_b$ in Agda. We also formalised Girard's and Gödel's embeddings which are embeddings from the standard call-by-name $\lambda$-calculus ($\lambda_n$) and call-by-value $\lambda$-calculus ($\lambda_v$) to the shared target, the call-by-box $\lambda$-calculus ($\lambda_b$). For both embeddings, we have proven some properties that are preserved in $\lambda_b$. We have proven the following propositions formally in Agda:

- Girard's embedding preserves reduction (Proposition 4.5).
- Girard's embedding preserves evaluation (Proposition 4.6).
- Gödel's embedding preserves reduction (Proposition 4.11 and 4.13).

To prove these propositions, we relied on several helper lemmas. The most important are Lemma 4.4 for the properties about Girard's translation, and Lemma 4.10 for the properties about Gödel's translation. These helper lemmas state that substitution is preserved and reflected by the translation. They were the most complex to prove, but they are fundamental for the proofs about preservation of reduction and evaluation.

We also saw that the formal definition of raise in Gödel's embedding is more complex than the less formal definition due to the use of the rename function. This rename function made proofs about Gödel's embedding significantly more complex than those about Girard's embedding, which is one reason why we did not prove the preservation of evaluation for Gödel's embedding formally.

In conclusion, this thesis provides a formal implementation for $\lambda_b$ in Agda. It contains the syntax, substitution, reduction and evaluation relations as well as some proofs of Girard's and Gödel's embeddings. The formalisation could serve as a basis for further research into the unification of call-by-name and call-by-value evaluation strategies in a modal setting, and could also serve as a foundation for future programming languages or proof assistants that natively support both evaluation strategies.

## 7.2 Future Work

This thesis has formalised the core functionalities of $\lambda_b$ and has proven some important properties of Girard's and Gödel's embeddings. However, there are still many areas for future work regarding this formalisation. We list some opportunities for future work below.

The formalisation of $\lambda_b$ can be extended with more features, such as case expressions, natural numbers or recursion. The book PLFA [33] implements these features in the standard $\lambda$-calculus, and they can be implemented in $\lambda_b$ as well.

Preservation of evaluation under Gödel's embedding has not been proven formally yet, but it is an interesting property to prove. It would make the formalisation more complete and could be useful for future research.

We can extend the formalisation with more properties, such as reflection of reduction and evaluation under Girard's and Gödel's embeddings. This would show that the target $\lambda_b$, does not simulate any other reductions or evaluations than those of $\lambda_n$ and $\lambda_v$. For example, the reflection of reduction under Girard's translation can be stated as follows: if $M^\circ \to_{\beta_b} N^\circ$, then $M \to_{\beta_n} N$ in $\lambda_n$.

We only formalised $\lambda_b$ in Agda, but it could be interesting to also formalise different frameworks that can simulate both cbn and cbv, such as the linear logic approach by Ehrhard and Giulio Guerrieri [14] or Maraist et al. [24], and the thunk-based approach by Hatcliff and Danvy [19]. All these formalisations could then be compared to see which is simpler, more efficient or more powerful.

The boxed $\lambda$-calculus can also be used to construct an interpreter for a programming language that supports both cbn and cbv evaluation. This is a more practical application and it could be interesting to see $\lambda_b$ in practice.

# Bibliography


[1] Agda Emacs Mode. `https://agda.readthedocs.io/en/v2.6.4/tools/emacs-mode.html`. Accessed: 2025-06-06.

[2] Agda Documentation. `https://agda.readthedocs.io/`, 2023. Accessed: 2025-06-14.

[3] Altenkirch, T., and Reus, B. Monadic presentations of lambda terms using generalized inductive types. In *Computer Science Logic* (Berlin, Heidelberg, 1999), J. Flum and M. Rodriguez-Artalejo, Eds., Springer Berlin Heidelberg, pp. 453–468.

[4] Ariola, Z. M., and Felleisen, M. The Call-by-Need Lambda Calculus. *Journal of Functional Programming 8*, 3 (1997), 265–301.

[5] Atten, v. M. The Development of Intuitionistic Logic. In *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds., Fall 2023 ed. Metaphysics Research Lab, Stanford University, 2023.

[6] Barendregt, H. P. Lambda Calculi with Types. In *Handbook of Logic in Computer Science*, S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, Eds., vol. 2. Oxford University Press, Oxford, 1992, pp. 117–309.

[7] Benton, N., Hur, C.-K., Kennedy, A. J., and McBride, C. Strongly Typed Term Representations in Coq. *Journal of Automated Reasoning 49*, 2 (2012), 141–159.

[8] Bierman, G. M., and Paiva, d. V. On an Intuitionistic Modal Logic. *Studia Logica 65*, 3 (2000), 383–416.

[9] Bruijn, d. N. G. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings) 75*, 5 (1972), 381–392.

[10] Church, A. An Unsolvable Problem of Elementary Number Theory. *American Journal of Mathematics 58*, 2 (1936), 345–363.

[11] Church, A. A Formulation of the Simple Theory of Types. *The Journal of Symbolic Logic 5*, 2 (1940), 56–68.

[12] Curry, H. B., and Feys, R. *Combinatory Logic, Volume I.* North-Holland Publishing Company, 1958.

[13] Curry, H. B., Feys, R., and Craig, W. *Combinatory Logic, Volume I.* North-Holland Publishing Company, 1958. Introduces substitution notation $[a/x]b$.

[14] Ehrhard, T., and Guerrieri, G. The Bang Calculus: An Untyped Lambda-Calculus Generalizing Call-by-Name and Call-by-Value. In *Proceedings of the 18th International Symposium on Principles and Practice of Declarative Programming (PPDP 2016)* (2016), ACM, pp. 174–187.

[15] Espírito Santo, J., Pinto, L., and Uustalu, T. Modal Embeddings and Calling Paradigms. In *4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019)* (Dagstuhl, Germany, 2019), H. Geuvers, Ed., vol. 131 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, pp. 18:1–18:20.

[16] Forster, Y., Schäfer, S., Spies, S., and Stark, K. Call-by-push-value in Coq: operational, equational, and denotational theory. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs* (New York, NY, USA, January 2019), CPP 2019, Association for Computing Machinery, pp. 118–131.

[17] Girard, J.-Y. Linear logic. *Theoretical Computer Science 50*, 1 (1987), 1–101.

[18] Gödel, K. Eine Interpretation des intuitionistischen Aussagenkalküls. In *Collected Works, Volume I: Publications 1929–1936*, S. Feferman, J. W. D. Jr., S. C. Kleene, G. H. Moore, R. M. Solovay, and J. van Heijenoort, Eds. Oxford University Press, New York, 1986, pp. 301–303. Originally published in *Ergebnisse eines mathematischen Kolloquiums*, Heft 4, 1933, pp. 39–40.

[19] Hatcliff, J., and Danvy, O. Thunks and the $\lambda$-calculus. *Journal of Functional Programming 7*, 3 (1997), 303–319.

[20] Heyting, A. Die formalen Regeln der intuitionistischen Logik. *Sitzungsberichte der Preußischen Akademie der Wissenschaften* (1930), 42–56.

[21] Hindley, J. R., and Seldin, J. P. *Lambda-Calculus and Combinators: An Introduction*, 2nd ed. Cambridge University Press, 2008.

[22] Howard, W. A. The Formulae-as-Types Notion of Construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. P. Seldin and J. R. Hindley, Eds. Academic Press, 1980, pp. 479–490.

[23] Levy, P. B. *Call-By-Push-Value*. Phd thesis, Queen Mary University of London, 2001.

[24] Maraist, J., Odersky, M., Turner, D. N., and Wadler, P. Call-by-name, call-by-value, call-by-need and the linear lambda calculus. *Theoretical Computer Science 228*, 1 (1999), 175–210.

[25] McKinsey, J. C. C., and Tarski, A. Some Theorems About the Sentential Calculi of Lewis and Heyting. *Journal of Symbolic Logic 13*, 1 (1948), 1–15.

[26] Nielson, H. R., and Nielson, F. *Semantics with Applications: A Formal Introduction*. John Wiley & Sons Ltd., 1999. Revised edition. Originally published in 1992 by John Wiley & Sons. Available online: `http://www.daimi.au.dk/~{}hrn`.

[27] Norell, U. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.

[28] Plotkin, G. D. Call-by-name, call-by-value and the $\lambda$-calculus. *Theoretical Computer Science 1*, 2 (1975), 125–159.

[29] Torrens, P. takanuva/cps. `https://github.com/takanuva/cps`, June 2025. Accessed: 2025-06-16.

[30] Troelstra, A. S. Introductory Note to 1933f. In *Collected Works, Volume I: Publications 1929–1936*, S. Feferman, J. W. D. Jr., S. C. Kleene, G. H. Moore, R. M. Solovay, and J. van Heijenoort, Eds. Oxford University Press, New York, 1986, pp. 296–299.

[31] Troelstra, A. S. *Lectures on Linear Logic*. CSLI Publications, Stanford, CA, 1992.

[32] Troelstra, A. S., and Dalen, v. D. *Constructivism in Mathematics: An Introduction, Volume I*. North-Holland, Amsterdam, 1988.

[33] Wadler, P., Kokke, W., and Siek, J. G. *De Bruijn Indices*. University of Edinburgh, Aug. 2022. Version 22.08, accessed 2025-06-06.

[34] Zdancewic, S. A. Zdancewic/linearity. `https://github.com/Zdancewic/linearity`, March 2019. Accessed: 2025-06-16. Includes contributions by Anton Relin.

[35] Zhao, J., Zhang, Q., and Zdancewic, S. A. Relational Parametricity for Polymorphic Linear Lambda Calculus. In *Asian Symposium on Programming Languages and Systems (APLAS)* (Shanghai, China, November 2010), vol. 6461 of *LNCS*, Springer, pp. 339–354.